PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ · ARQUITETURA E SOLUÇÕES EM CLOUD

Documentação de Arquitetura.

Veterano. — Plataforma de mentoria entre veteranos e calouros. Documentação completa no padrão arc42, com C4 Model (níveis 1 a 4), UML, DER e Software Architecture Canvas, cobrindo os estilos Serverless, Microfrontend, BFF + Microsserviços + Database per Service, API Gateway, Arquitetura Orientada a Eventos e Clean Architecture com Vertical Slice.

GRUPO

Bento Barp

Guilherme Reis

Guilherme Selenko

Vinicius Trevisan

Repositório: github.com/ViniMTrevisan/pjbl-mentoria-cloud · código desta entrega em `api/src` e `frontend/src`
Aplicação publicada: white-rock-03ee36c10.7.azurestaticapps.net · API: func-pjbl-mentoria.azurewebsites.net
Versão 2.0 · outubro de 2026 · documentação completa

Sumário

1. Introdução e Objetivos

2. Restrições da Arquitetura

3. Escopo e Contexto do Sistema — *C4 Nível 1*

4. Estratégia de Solução — *estilos arquiteturais e Software Architecture Canvas*

5. Visão de Blocos de Construção — *C4 Níveis 2, 3 e 4 (UML Classes e Componentes), Clean Architecture e Vertical Slice*

6. Visão de Runtime — *UML Sequência e Máquina de Estados*

7. Visão de Implantação — *UML Deployment*

8. Conceitos Transversais — *EDA, Database per Service e DER*

9. Decisões de Arquitetura — *ADRs*

10. Requisitos de Qualidade

11. Riscos e Dívidas Técnicas

A. Apêndice — Glossário

Como ler as marcações. O documento descreve a arquitetura-alvo do Veterano. e indica, em cada elemento, o quanto dela já existe: IMPLEMENTADO está publicado em Azure; PARCIAL existe em código, mas ainda não na forma final do estilo (por exemplo, as telas que serão microfrontends hoje convivem no mesmo bundle); PLANEJADO está desenhado e ainda não construído. Nos diagramas C4, a **borda grossa** marca os contêineres cujo papel já é cumprido por código publicado. A Clean Architecture e o Vertical Slice (seção 5.3) descrevem o código entregue em `api/src`, arquivo por arquivo.

Seções. As onze seções numeradas seguem o template oficial arc42 (arc42.org). O glossário, que o template traz como seção 12, foi mantido como apêndice.

1. Introdução e Objetivos

Veterano. é uma plataforma web que conecta calouros que precisam de ajuda em uma matéria a veteranos dispostos a dar mentoria. Hoje esse encontro acontece de forma informal — grupos de WhatsApp, indicação pessoal, sorte — e depende de o calouro já conhecer alguém. O sistema cobre o fluxo inteiro: **cadastro** → **descoberta** → **agendamento** → **notificação** → **avaliação**.

1.1 Visão geral dos requisitos

ID	REQUISITO FUNCIONAL	DONO NA ARQUITETURA	SITUAÇÃO
RF1	Cadastro e login do usuário.	Entra External ID + Shell	PLANEJADO
RF2	Definir o perfil como mentor, mentorado ou ambos.	Serviço de Mentores	PLANEJADO
RF3	Mentor cadastra as matérias/áreas em que pode ajudar.	Serviço de Mentores · MFE Perfil	PARCIAL CRUD em MongoDB
RF4	Mentor define sua disponibilidade de horários.	Serviço de Mentores · MFE Perfil	PLANEJADO
RF5	Mentorado busca mentores por curso, matéria ou área.	Serviço de Mentores · MFE Descoberta	IMPLEMENTADO sobre dados mock
RF6	Mentorado solicita uma sessão.	Serviço de Sessões · MFE Agenda	PLANEJADO
RF7	Mentor aceita ou recusa a solicitação.	Serviço de Sessões · MFE Agenda	PLANEJADO
RF8	Notificar o usuário a cada mudança de status.	Serviço de Notificações (eventos)	PLANEJADO
RF9	Visualizar sessões futuras e histórico.	Serviço de Sessões · MFE Agenda	IMPLEMENTADO sobre dados mock
RF10	Mentorado avalia o mentor após a sessão concluída.	Serviço de Avaliações	PLANEJADO
RF11	Calcular e exibir a reputação do mentor.	Domínio de Mentores (projeção)	IMPLEMENTADO cálculo na leitura
RF12	Marcar a sessão como concluída ou cancelada.	Serviço de Sessões	PLANEJADO

Ficam **fora de escopo**: chat em tempo real, videochamada integrada, pagamento de mentorias, app mobile nativo e integração com o sistema acadêmico da instituição.

1.2 Metas de qualidade

#	ATRIBUTO	META CONCRETA
1	Manutenibilidade	Um caso de uso novo ou alterado mexe em uma fatia vertical e, no máximo, no domínio. Trocar o banco de um serviço muda apenas o adaptador de saída.
2	Implantabilidade	Cada microfrontend e cada microsserviço é publicado sozinho, sem republicar os demais.
3	Confiabilidade	A falha de um serviço não derruba os outros: notificação e reputação são assíncronas, e o BFF devolve resposta parcial quando um serviço não responde.
4	Segurança	Token validado no gateway antes de qualquer função executar; autorização por recurso dentro do serviço dono do dado.
5	Custo	Tudo serverless, com custo ocioso próximo de zero e total dentro do crédito Azure for Students.
6	Usabilidade	O calouro conclui a busca por mentor no celular, sem zoom; interface em pt-BR, datas em `dd/MM/yyyy`, fuso `America/Sao_Paulo`.

1.3 Stakeholders

PAPEL	QUEM É	O QUE ESPERA DA ARQUITETURA
Calouro (mentorado)	Aluno de 1º a 3º período	Achar rápido quem já passou pela matéria e agendar pelo celular.
Veterano (mentor)	Aluno a partir do 4º período	Controlar disponibilidade, ser avisado de solicitações e construir reputação visível.
Coordenação do curso	Professores e coordenadores	Moderar cadastros e ver métricas sem acesso direto aos bancos.
Equipe de desenvolvimento	Bento Barp, Guilherme Reis, Guilherme Selenko, Vinicius Trevisan	Trabalhar em paralelo — cada um dono de um MFE ou serviço — com contratos estáveis entre as partes.
Professor da disciplina	Avaliador do PJBL	Ver cada estilo arquitetural aplicado com justificativa, rastreável até o código e os requisitos.

2. Restrições da Arquitetura

2.1 Restrições técnicas

RESTRIÇÃO	ORIGEM E CONSEQUÊNCIA
Hospedagem em Microsoft Azure	Exigência da disciplina. Toda a seleção de serviços da seção 4 parte do catálogo Azure; a única exceção é o MongoDB Atlas, já usado pela entrega de CRUD.
Front-end em Azure Static Web Apps	Artefato estático, sem renderização no servidor. A composição dos microfrontends precisa ser feita no navegador (Module Federation), não no servidor.
Back-end em Azure Functions	Funções stateless que toleram *cold start*. Nada de estado em memória entre requisições — exceto o cliente MongoDB reaproveitado entre invocações, que é cache de conexão, não de dados.
Assinatura Azure for Students	Crédito limitado e sem cartão: só SKUs gratuitas, de consumo ou serverless (APIM Consumption, Service Bus Standard, Azure SQL serverless, Cosmos DB serverless).
Regiões permitidas por policy	Apenas `centralus`, `mexicocentral`, `southafricanorth`, `canadacentral` e `chilecentral`. `brazilsouth` está bloqueada.
MongoDB Atlas M0	Camada gratuita compartilhada: 512 MB e limite de conexões. Obriga a reaproveitar o cliente entre invocações (`infrastructure/mongo/client.js`).
Uso de IA generativa	O código gerado é rastreável aos prompts versionados (`Prompt.md`, `docs/TDE2-PROMPTS.md`) e revisado antes de publicar.

2.2 Restrições organizacionais e convenções

RESTRIÇÃO	CONSEQUÊNCIA
Quatro alunos, um semestre	A arquitetura-alvo é distribuída, mas a construção é incremental: primeiro um monólito modular bem fatiado, depois a extração de serviços e MFEs pelas costuras já existentes (seção 9, ADR-01 e ADR-04).
Sem orçamento para licenças	Ferramental gratuito ou open source: Vite, React, Node, Mermaid, GitHub.
Nomes de domínio em português	Funções e campos em pt-BR (`alterarMentor`, `camposAlterados`), reduzindo a tradução entre requisito e código. Nomes de arquivo das fatias seguem o verbo em inglês (`updateMentor.js`), como já está no repositório.
Repositório público	Nenhum segredo versionado. `MONGODB_URI` fica nas configurações da Function App e, na evolução, no Key Vault.

3. Escopo e Contexto do Sistema

> A fronteira do sistema: quem usa o Veterano. e de quais sistemas externos ele depende. O diagrama é o **Nível 1 do C4 Model** (contexto). Os bancos de cada serviço — inclusive o MongoDB Atlas, que é um serviço gerenciado — aparecem como contêineres no Nível 2, porque guardam dados do próprio sistema.

3.1 Contexto de negócio

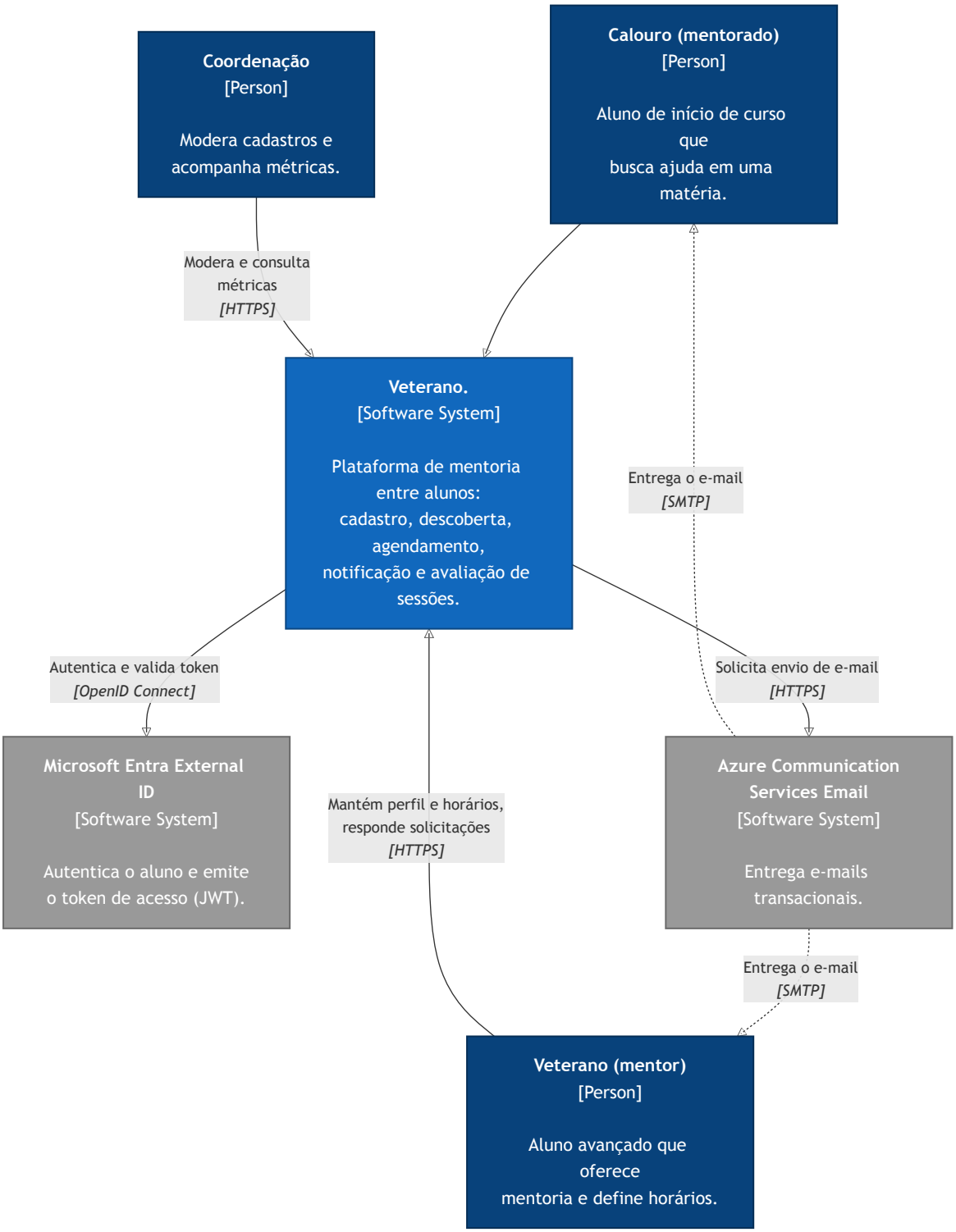


Figura 1. C4 Model — Nível 1 (Contexto do Sistema). Pessoas, o sistema Veterano. e os sistemas externos com que ele se comunica.

PARCEIRO	ENTRADA NO SISTEMA	SAÍDA DO SISTEMA
Calouro	Filtros de busca, solicitação de sessão, nota e comentário.	Mentores com reputação, situação das solicitações, notificações.
Veterano	Perfil, áreas, horários, aceite ou recusa.	Solicitações recebidas, agenda, reputação acumulada.
Coordenação	Ações de moderação.	Métricas de uso.
Entra External ID	Credenciais no fluxo de login.	Token JWT, validado pelo API Gateway.
ACS Email	Destinatário, assunto e corpo.	Aceite do envio; a entrega é assíncrona.

3.2 Contexto técnico

CANAL	PROTOCOLO	OBSERVAÇÃO
Navegador → Static Web Apps	HTTPS	Shell e remotes servidos pela borda. **IMPLEMENTADO** (bundle único)
Navegador → API Gateway	HTTPS, JSON, Bearer	Ponto único de entrada da API. PLANEJADO — hoje o navegador chama a Function App diretamente, com CORS restrito.
Shell → Entra External ID	OIDC + PKCE	Cliente público, sem segredo. PLANEJADO
Serviço de Mentores → MongoDB Atlas	MongoDB wire sobre TLS	Driver `mongodb` 7, conexão aberta sob demanda e reaproveitada. **IMPLEMENTADO**
Serviços ↔ Service Bus	AMQP 1.0 (5671)	Publicação e assinatura de eventos de domínio. PLANEJADO
Serviço de Notificações → ACS	HTTPS	Envio de e-mail. PLANEJADO
Functions → Application Insights	HTTPS	Requisições, dependências e exceções. **IMPLEMENTADO**

4. Estratégia de Solução

> As decisões estruturais que moldam a solução. A primeira tabela mostra onde cada estilo arquitetural exigido aparece e por quê; as decisões detalhadas estão como ADRs na seção 9.

4.1 Estilos arquiteturais adotados

ESTILO	ONDE APARECE NO VETERANO.	POR QUE FOI ESCOLHIDO	SITUAÇÃO
Serverless	Todo o cômputo em Azure Functions (Flex Consumption); front-ends em Static Web Apps; APIM Consumption, Service Bus, Azure SQL serverless, Cosmos DB serverless.	Tráfego intermitente, com pico em época de prova. Pagar por execução e escalar a zero cabe no crédito de estudante e elimina gestão de servidor.	IMPLEMENTADO
Microfrontend	Shell (host) + três remotes: Descoberta, Agenda e Perfil do Mentor, compostos no navegador por Module Federation. Seções 5.1 e 5.2.1.	Cada aluno pode ser dono de uma área do produto e publicá-la sozinho. As fronteiras dos MFEs coincidem com as dos serviços, então cada time vai do botão ao banco.	PARCIAL telas separadas, mesmo bundle
BFF	BFF Web em Functions, um endpoint por tela, agregando os microsserviços. Seção 5.2.2.	A tela de perfil precisa de dados de Mentores, Avaliações e disponibilidade; sem BFF o navegador faria três chamadas e juntaria os dados.	PARCIAL respostas já moldadas à tela
Microsserviços	Mentores, Sessões, Avaliações e Notificações — um por subdomínio, cada um com sua Function App.	Os subdomínios mudam em ritmos diferentes e têm cargas diferentes; separar permite escalar e publicar cada um sem afetar os outros.	PARCIAL Mentores existe
Database per Service	MongoDB Atlas (Mentores), Azure SQL (Sessões), Cosmos DB (Avaliações), Table Storage (Notificações). Seção 8.2.	Nenhum serviço lê a tabela de outro; o acoplamento fica no contrato da API e dos eventos. Cada banco foi escolhido pela forma do dado.	PARCIAL BD Mentores existe
API Gateway	Azure API Management Consumption na frente do BFF: JWT, rate limit, CORS, versionamento. Seção 5.2.2.	Concentra preocupações transversais em um lugar, antes de qualquer função rodar, e esconde a topologia interna do cliente.	PLANEJADO
Orientada a Eventos (EDA)	Tópico `veterano.eventos` no Service Bus; outbox no Serviço de Sessões; consumidores idempotentes. Seções 6.2 e 8.1.	Notificação e reputação são efeitos colaterais de uma sessão: não devem atrasar nem derrubar a requisição que os origina, e precisam ser entregues mesmo com o consumidor fora do ar.	PLANEJADO
Clean Architecture + Vertical Slice	Código de `api/src`: `functions/` (entrada), `features/` (fatias), `domain/` e `ports/`, `infrastructure/` (saída). Seção 5.3.	Organização por caso de uso facilita achar e mudar uma funcionalidade; a regra de dependência protege o domínio de Azure e MongoDB e permite trocar o mock pelo banco sem tocar nas regras.	IMPLEMENTADO

4.2 Decisões-chave

DECISÃO	ESCOLHA	POR QUÊ
Construção incremental	Monólito modular primeiro, extração depois	As costuras dos serviços já existem no código como fatias e portas; extrair é mover pastas e trocar a chamada local por HTTP ou evento.
Comunicação síncrona	Só na leitura (BFF → serviços)	Leituras precisam de resposta imediata. Escritas que afetam outro serviço viram evento.
Consistência entre serviços	Eventual, por eventos	Transação distribuída não cabe em serverless. A reputação pode atrasar segundos sem prejuízo ao usuário.
Entrega de eventos	Outbox transacional + pelo menos uma vez + consumidor idempotente	Evita o caso “sessão gravada, evento perdido” sem depender de transação entre banco e barramento.
Identidade	Entra External ID, validado no gateway	Senha não é guardada pelo sistema; token rejeitado não chega a gastar execução de função.
Regra de negócio	Só no servidor, dentro do domínio	O front-end não recalcula reputação nem decide transição de estado.

4.3 Software Architecture Canvas

CONTEXTO E PROBLEMA

- Calouros não têm canal estruturado para tirar dúvidas de matéria.
- Veteranos dispostos a ajudar não têm onde se oferecer nem acumular reputação.
- O encontro depende de rede pessoal e é invisível para a coordenação.

OBJETIVOS DE NEGÓCIO

- Reduzir evasão nos primeiros períodos.
- Dar visibilidade a quem ajuda.
- Tornar a mentoria mensurável.

USUÁRIOS E STAKEHOLDERS

- Calouro — usuário primário.
- Veterano — oferta.
- Coordenação — moderação.
- Equipe de 4 alunos.

REQUISITOS-CHAVE

- RF5 descoberta com filtros.
- RF6/RF7 solicitação e resposta.
- RF8 notificação por evento.
- RF10/RF11 avaliação e reputação.

ATRIBUTOS DE QUALIDADE

- Manutenibilidade por fatia.
- Deploy independente.
- Falha isolada por serviço.
- Custo ocioso perto de zero.

RESTRIÇÕES

- Azure + SWA + Functions.
- Assinatura de estudante.
- `brazilsouth` bloqueada.
- Atlas M0 (512 MB).
- Repositório público.

ESTILOS

- Serverless.
- Microfrontends.
- API Gateway + BFF.
- Microsserviços + DB per Service.
- EDA.
- Clean Architecture + Vertical Slice.

PRINCÍPIOS

- Dependências apontam para o domínio.
- Um caso de uso, uma fatia.
- Nenhum serviço lê o banco de outro.
- Escrita entre serviços é evento.
- Complexidade só com requisito.

COMPONENTES

- Shell + 3 MFEs.
- API Gateway, BFF Web.
- 4 microsserviços.
- Barramento de eventos.
- 4 bancos.

TECNOLOGIAS

- React 19, Vite, TypeScript.
- Azure Functions v4, Node 22.
- API Management, Service Bus.
- MongoDB Atlas, Azure SQL, Cosmos DB, Table Storage.

RISCOS

- Complexidade distribuída para 4 pessoas.
- *Cold start* em cadeia.
- Consistência eventual visível.
- Crédito de estudante.

MÉTRICAS DE SUCESSO

- **Produto:** cadastrar → publicar disponibilidade → buscar → solicitar → aceitar → concluir → avaliar → ver reputação executa de ponta a ponta.
- **Arquitetura:** busca em até 2 s no P95; falha de e-mail não gera erro HTTP; nenhuma alteração de requisito exige mudança em mais de um serviço.
- **Operação:** custo mensal dentro do crédito; cada serviço publicável sozinho; nenhuma mensagem perdida (dead-letter monitorada).

5. Visão de Blocos de Construção

> A decomposição estática, em três níveis de refinamento. Nível 1 = **C4 Nível 2** (contêineres); Nível 2 = **C4 Nível 3** (componentes de cada contêiner); Nível 3 = **C4 Nível 4** (código), com diagramas UML de classes e de componentes e a organização Clean Architecture + Vertical Slice do código entregue.

5.1 Nível 1 — caixa-branca do sistema (C4 Nível 2: Contêineres)

O diagrama mostra os seis estilos ao mesmo tempo: os **microfrontends** no topo, o **API Gateway** como entrada única, o **BFF** agregando, os **microsserviços** cada um com **seu banco**, e o **barramento de eventos** ligando os serviços sem chamada direta. Tudo roda em serviços **serverless**.

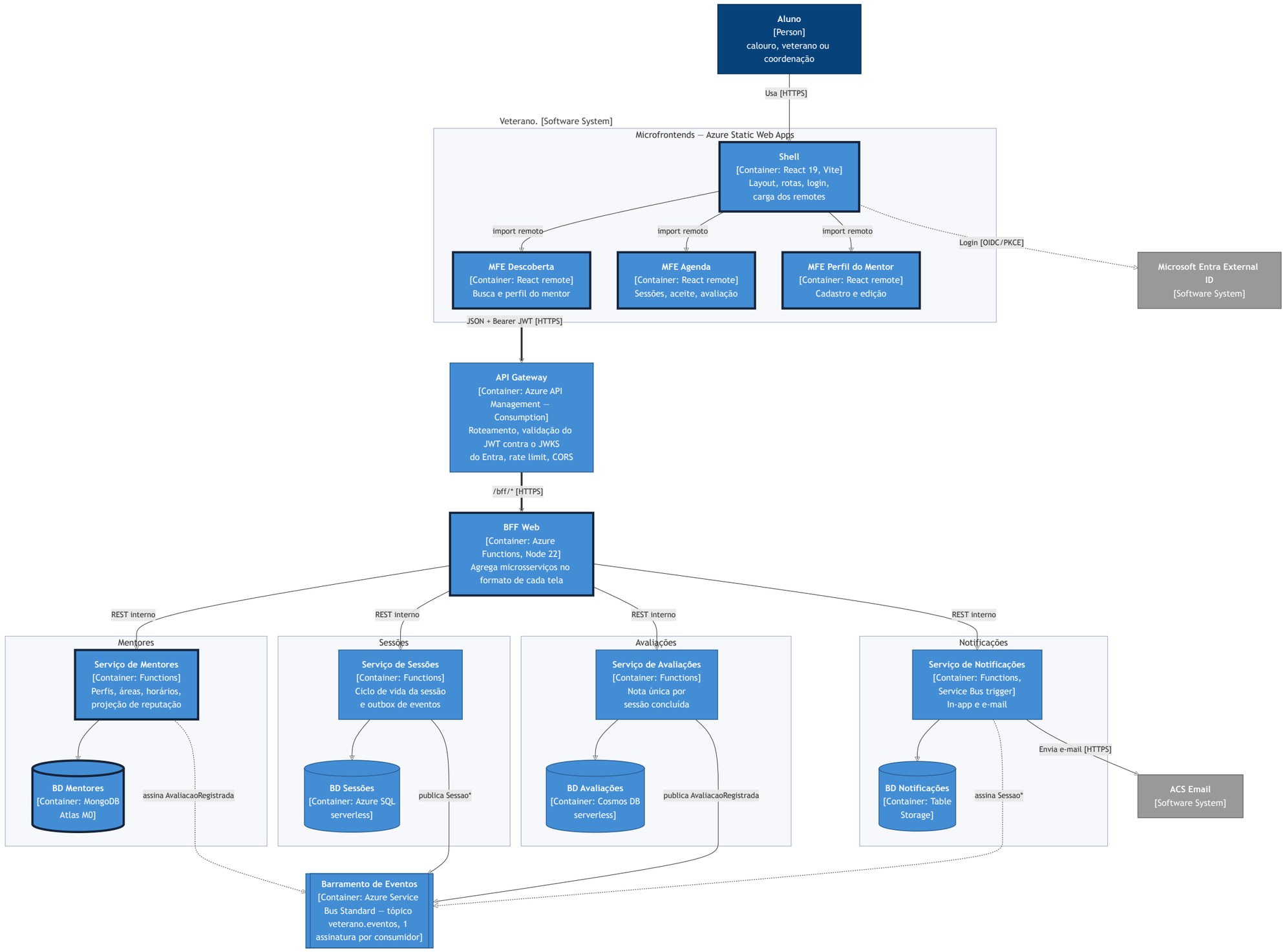


Figura 2. C4 Model — Nível 2 (Contêineres). Borda grossa: papel já cumprido por código publicado (em alguns casos ainda no mesmo artefato). Setas grossas: caminho de toda requisição do usuário.

CONTÊINER	TECNOLOGIA	RESPONSABILIDADE	SITUAÇÃO
Shell	React 19, Vite, SWA	Layout, navegação, login e carga dos remotes. Hoje é o `App.tsx` ("casca") da SPA.	PARCIAL
MFE Descoberta	React remote	Busca de mentores e perfil público (`Mentores.tsx`, `MentorDetalhe.tsx`).	PARCIAL
MFE Agenda	React remote	Sessões futuras e histórico (`Sessoes.tsx`); aceite e avaliação no alvo.	PARCIAL
MFE Perfil do Mentor	React remote	Cadastro e edição do mentor (`Cadastro.tsx`).	PARCIAL
API Gateway	API Management Consumption	Entrada única: JWT, rate limit, CORS, roteamento e versão.	PLANEJADO
BFF Web	Azure Functions	Um endpoint por tela, agregando os serviços. Hoje os endpoints públicos da Function App já devolvem dados no formato da tela (ex.: `proximas` / `historico`).	PARCIAL
Serviço de Mentores	Azure Functions + MongoDB Atlas	Perfil, áreas, horários e projeção de reputação. É a Function App `func-pjbl-mentoria`.	IMPLEMENTADO
Serviço de Sessões	Azure Functions + Azure SQL	Máquina de estados da sessão e outbox de eventos.	PLANEJADO
Serviço de Avaliações	Azure Functions + Cosmos DB	Uma avaliação por sessão concluída; publica `AvaliacaoRegistrada`.	PLANEJADO
Serviço de Notificações	Azure Functions + Table Storage	Consome eventos de sessão; grava a notificação in-app e envia e-mail.	PLANEJADO
Barramento de Eventos	Service Bus Standard	Tópico com uma assinatura filtrada por consumidor e dead-letter.	PLANEJADO

5.2 Nível 2 — componentes dos contêineres (C4 Nível 3)

5.2.1 Microfrontends: Shell e remotes

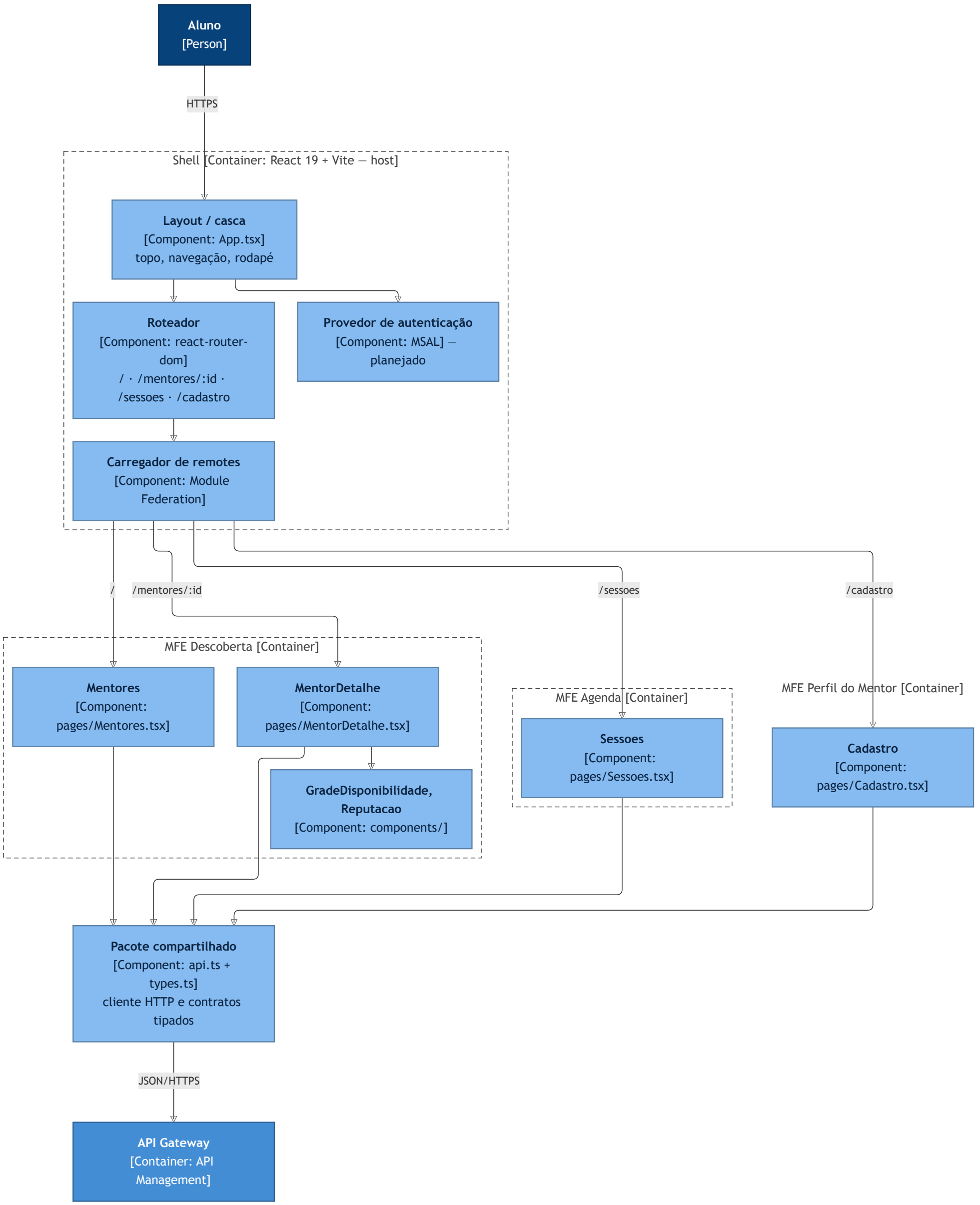


Figura 3. C4 Model — Nível 3 (Componentes) do front-end. O Shell carrega cada remote pela rota; todos compartilham o cliente HTTP e os tipos.

COMPONENTE	CÓDIGO HOJE	NA FORMA DE MICROFRONTEND
Layout / casca	`frontend/src/App.tsx`	Fica no Shell. Navegação e rodapé comuns a todos os remotes.
Roteador	`react-router-dom` em `App.tsx`	Fica no Shell; cada rota passa a carregar um remote via `import()` do Module Federation.
Páginas	`frontend/src/pages/*.tsx`	Cada pasta vira um projeto Vite com seu próprio deploy em SWA.
Pacote compartilhado	`api.ts` , `types.ts`	Publicado como pacote interno e declarado `shared` (singleton) junto com React, para não carregar duas cópias.

5.2.2 API Gateway e BFF Web

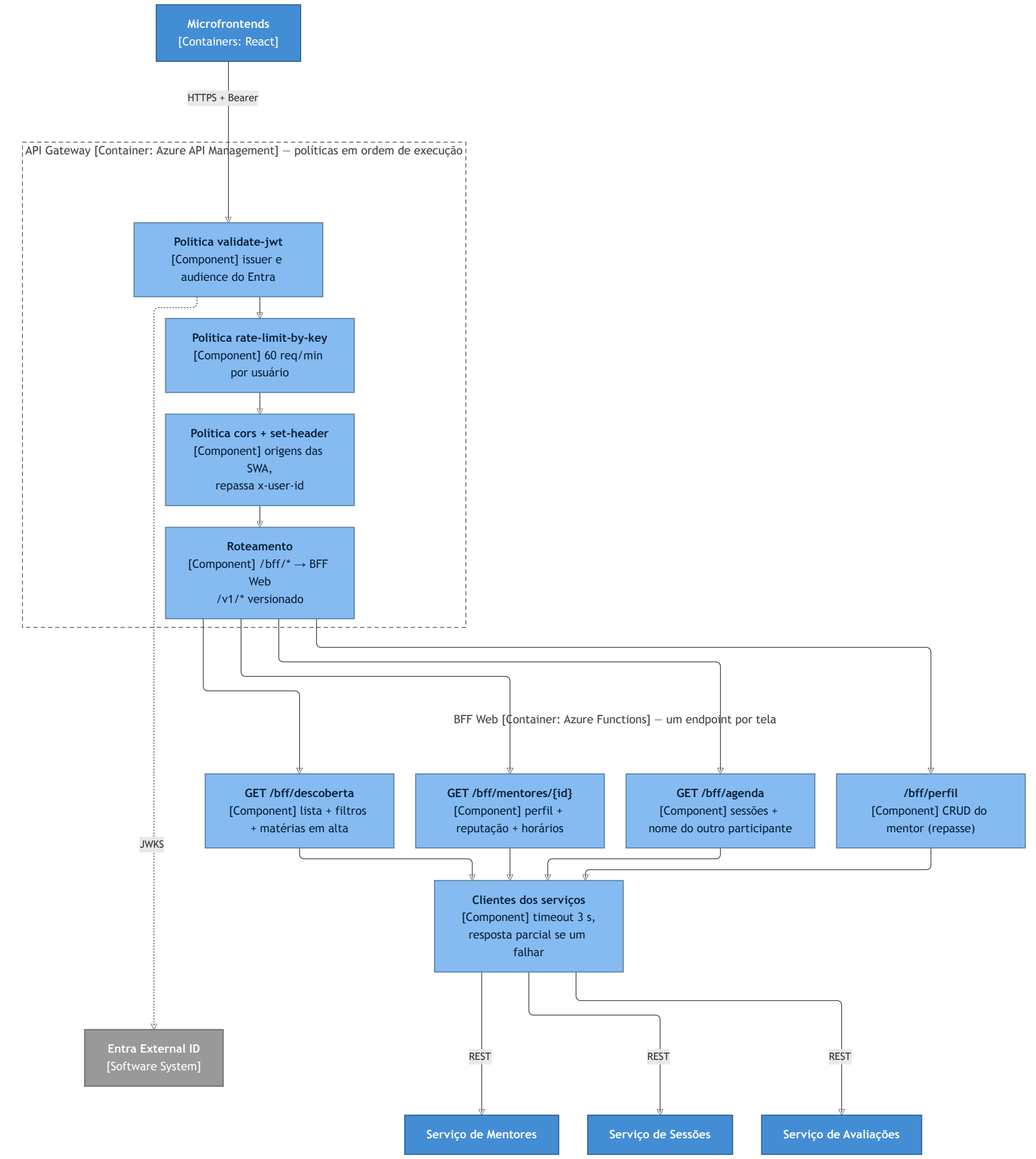


Figura 4. C4 Model — Nível 3 (Componentes) do API Gateway e do BFF Web. As políticas do gateway executam em ordem antes de chegar ao BFF.

O gateway resolve o que é igual para toda rota (quem é o usuário, quanto ele pode chamar, de qual origem) e o BFF resolve o que é específico de cada tela (quais serviços chamar e como juntar as respostas). O BFF não tem banco nem regra de negócio: se um serviço não responder em 3 s, devolve a parte que obteve e sinaliza o campo ausente, em vez de falhar a tela inteira.

5.2.3 Serviço de Mentores — código entregue

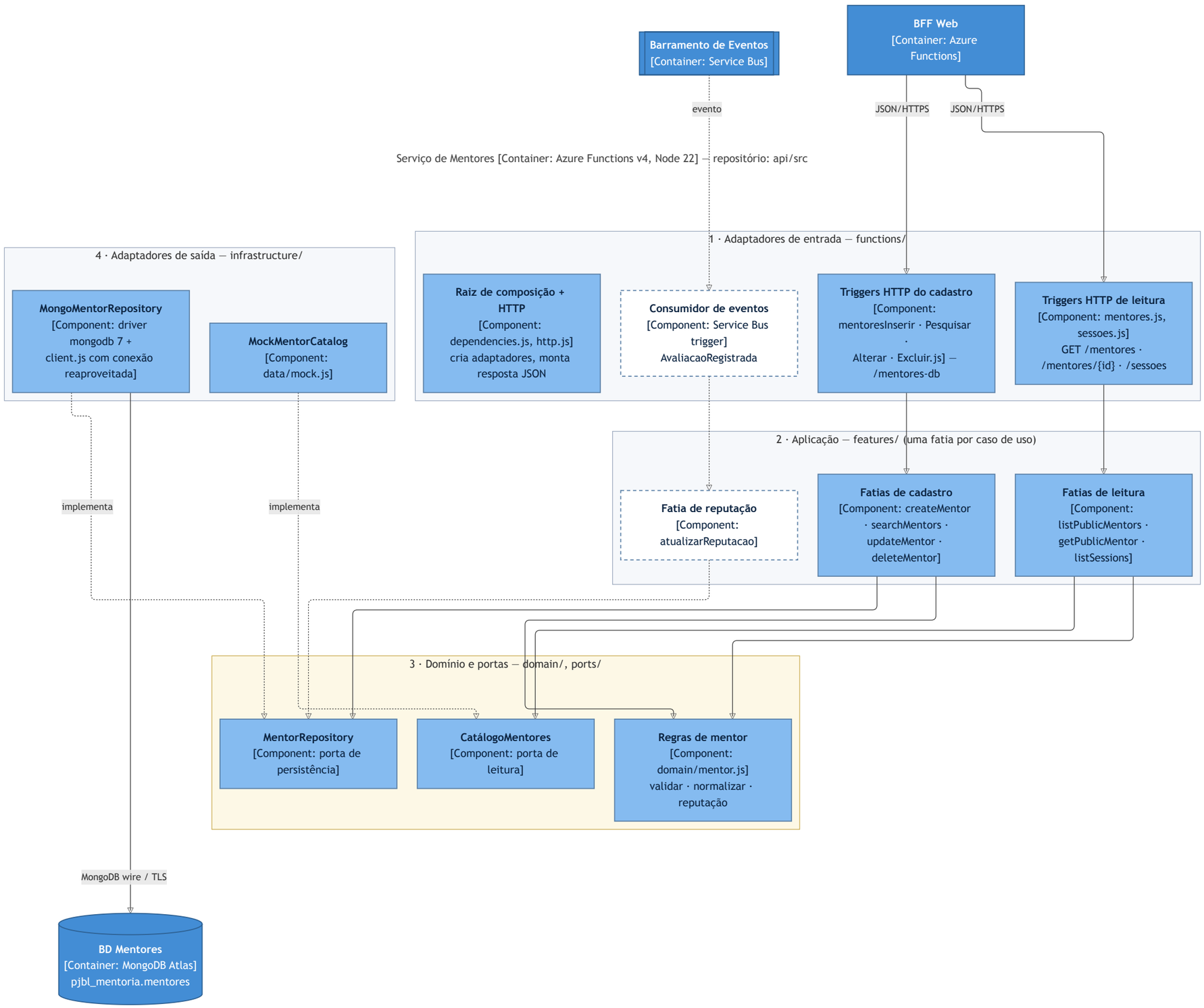


Figura 5. C4 Model — Nível 3 (Componentes) do Serviço de Mentores, espelhando as pastas de api/src. Componentes tracejados são planejados.

COMPONENTE	ARQUIVOS	RESPONSABILIDADE
Triggers HTTP de leitura	`functions/mentores.js`, `sessoes.js`	Lêem query e rota e chamam a fatia correspondente. Não contêm regra.
Triggers HTTP do cadastro	`functions/mentoresInserir|Pesquisar|Alterar|Excluir.js`	Uma Azure Function por operação do CRUD em `/api/mentores-db`.
Raiz de composição + HTTP	`functions/dependencies.js`, `http.js`	Cria os adaptadores concretos uma vez e os injeta; padroniza resposta JSON e converte exceção em 500 sem expor detalhes do driver.
Fatias	`features/mentors/*.js`, `features/sessions/*.js`	Um arquivo por caso de uso; devolve `{status, body}`, sem conhecer Azure Functions.
Domínio	`domain/mentor.js`	Funções puras: validação, normalização, campos alterados e cálculo de reputação.
Portas	`ports/MentorRepository.js`, `ports/CatalogoMentores.js`	Contratos estruturais (JSDoc) que as fatias exigem.
Adaptadores de saída	`infrastructure/mongo/*`, `infrastructure/mock/*`	Implementam as portas sobre MongoDB Atlas e sobre os dados fictícios.

5.2.4 Serviço de Sessões

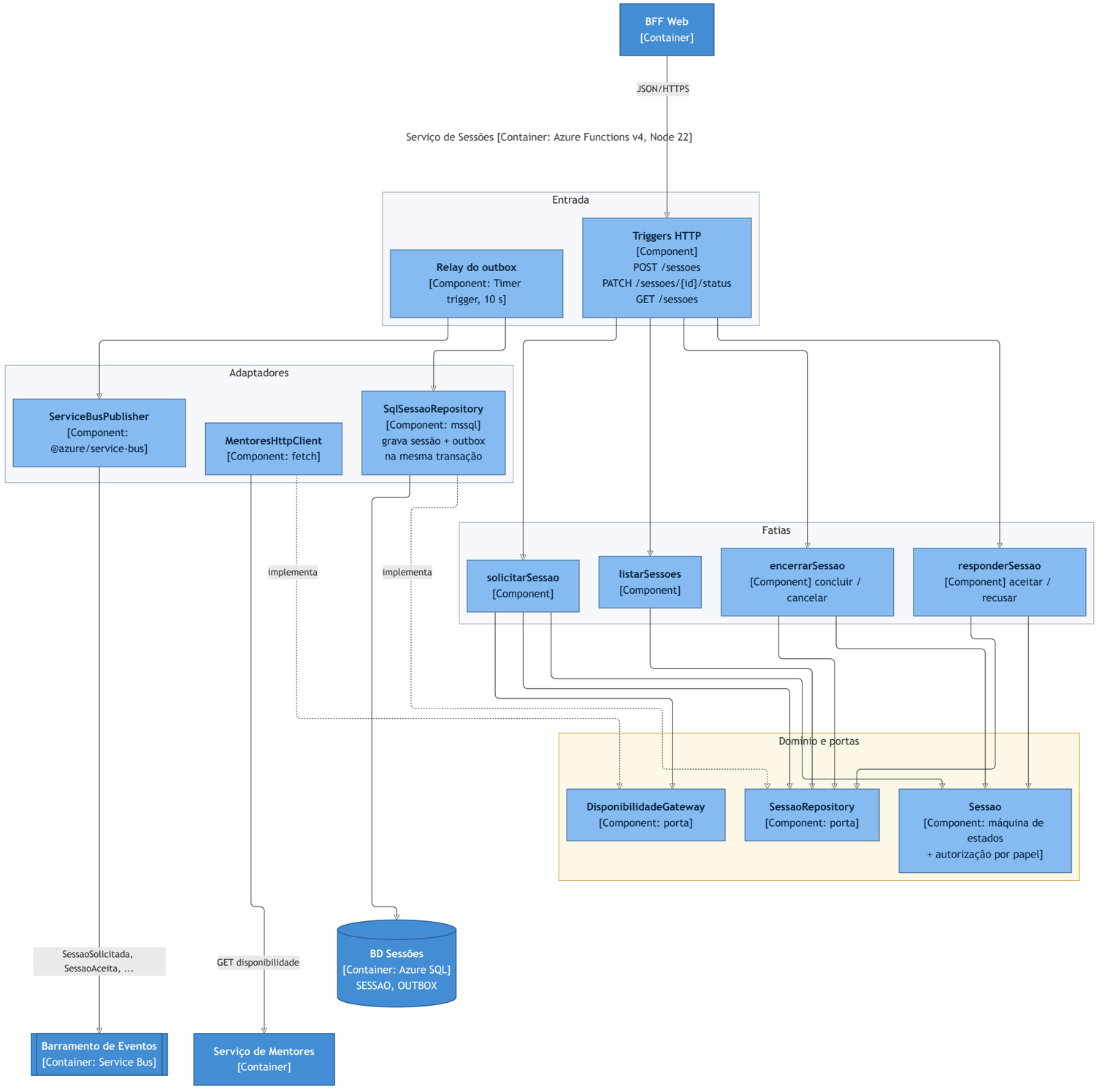


Figura 6. C4 Model — Nível 3 (Componentes) do Serviço de Sessões (planejado), com a mesma organização do Serviço de Mentores e o outbox de eventos.

O serviço repete a estrutura do código entregue: triggers, fatias, domínio com portas e adaptadores. A diferença está no lado de saída: o `SqlSessaoRepository` grava a sessão e a linha de `OUTBOX_EVENTO` na mesma transação, e um *timer trigger* separado publica o que estiver pendente no Service Bus. A consulta de disponibilidade ao Serviço de Mentores passa por uma porta (`DisponibilidadeGateway`), então o domínio não sabe que existe HTTP.

5.2.5 Serviços de Avaliações e de Notificações

SERVIÇO	FATIAS	ENTRADA E SAÍDA
Avaliações	`registrarAvaliacao` , `listarAvaliacoesDoMentor` , `liberarAvaliacao`	Recebe `SessaoConcluida` e libera a avaliação; HTTP para registrar; publica `AvaliacaoRegistrada` . Cosmos DB com `sessaoId` como chave única.
Notificações	`notificarMudancaDeSessao` , `listarNotificacoes` , `marcarComoLida`	Service Bus trigger na assinatura `notificacoes` ; grava em Table Storage (partição = usuário); envia e-mail *best effort* pelo ACS.

5.3 Nível 3 — código (C4 Nível 4)

5.3.1 Clean Architecture e Vertical Slice no código entregue

O back-end entregue (`api/src`) combina os dois estilos. A **Clean Architecture** define camadas e a regra de que dependências só apontam para dentro; o **Vertical Slice** define que cada caso de uso é um arquivo próprio que atravessa essas camadas, em vez de existir um “service” genérico com todos os métodos.

```
api/src/
├── functions/              ← adaptadores de entrada (Azure Functions) — camada externa
│   ├── mentores.js           GET /mentores, GET /mentores/{id}
│   ├── sessoes.js            GET /sessoes
│   ├── mentoresInserir.js    POST   /mentores-db
│   ├── mentoresPesquisar.js  GET    /mentores-db
│   ├── mentoresAlterar.js    PUT|PATCH /mentores-db/{id}
│   ├── mentoresExcluir.js    DELETE /mentores-db/{id}
│   ├── health.js             GET /health
│   ├── http.js               json(), lerCorpo(), executarCasoHttp()
│   └── dependencies.js       raiz de composição: cria e injeta os adaptadores
├── features/               ← aplicação: uma fatia por caso de uso
│   ├── mentors/  listPublicMentors · getPublicMentor · createMentor
│   │             searchMentors · updateMentor · deleteMentor
│   └── sessions/ listSessions
├── domain/mentor.js        ← regras puras: calcularReputacao, validarMentor, novoMentor, camposAlterados
├── ports/                  ← contratos exigidos pelas fatias
│   ├── MentorRepository.js
│   └── CatalogoMentores.js
├── infrastructure/         ← adaptadores de saída
│   ├── mongo/  MongoMentorRepository.js · client.js
│   └── mock/   MockMentorCatalog.js
└── data/mock.js            ← fixtures, sem lógica
```

Regra de dependência, verificável no código: `domain/mentor.js` não importa nada; as fatias importam só o domínio e recebem portas como parâmetro; apenas `infrastructure/mongo` importa o pacote `mongodb` ; apenas `functions/` importa `@azure/functions` . Quem liga as peças é `dependencies.js` , na borda. A figura a seguir mostra as fatias (colunas) cruzando as camadas (faixas).

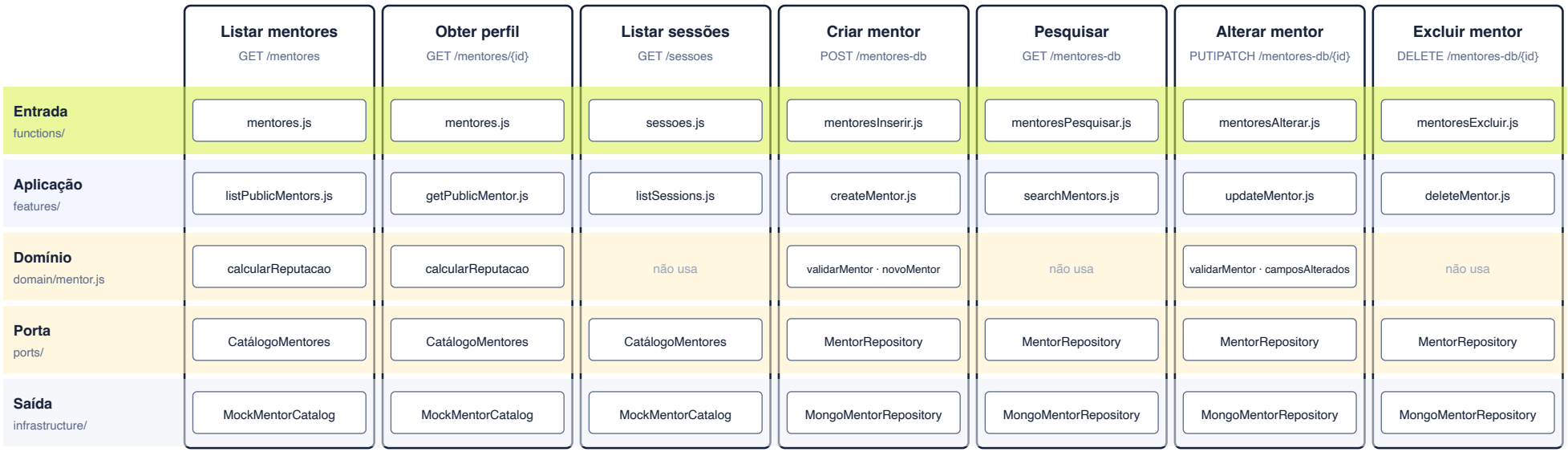


Figura 7. Vertical Slice × Clean Architecture no código entregue. Cada coluna é um caso de uso e mostra o arquivo que o implementa em cada camada.

PRINCÍPIO SOLID	ONDE APARECE
Responsabilidade única	Trigger cuida de HTTP; fatia coordena um caso de uso; `mentor.js` guarda regras; repositório executa persistência.
Aberto/fechado	Novo caso de uso = nova fatia + novo trigger, sem editar as existentes.
Substituição de Liskov	`MockMentorCatalog` e um futuro catálogo sobre Mongo cumprem o mesmo `CatalogoMentores`; a fatia não percebe a troca.
Segregação de interfaces	Leitura pública depende de `CatalogoMentores` (3 métodos); cadastro depende de `MentorRepository` (6 métodos). Ninguém recebe métodos que não usa.
Inversão de dependência	Fatias recebem `repository` / `catalogo` por parâmetro; `dependencies.js` escolhe a implementação concreta.

5.3.2 Diagrama de classes (UML)

JavaScript não declara interfaces; os contratos são `@typedef` JSDoc e os módulos exportam funções. O diagrama representa cada módulo como classe UML com estereótipo (`«use case»` , `«interface»` , `«adapter»` , `«module»`), e cada operação corresponde a uma função exportada com a mesma assinatura.

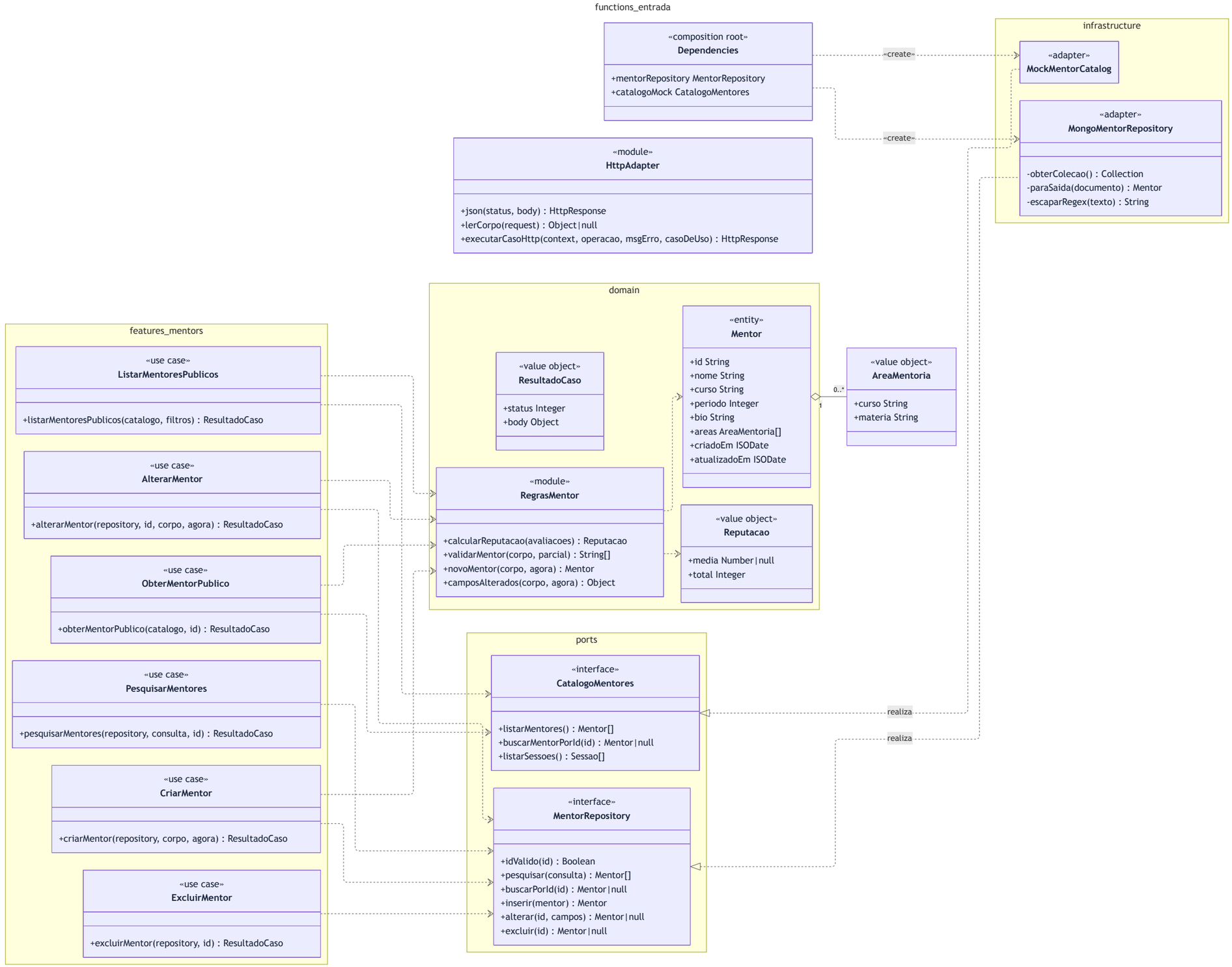


Figura 8. C4 Model — Nível 4 (Código). Diagrama de classes UML do Serviço de Mentores, agrupado pelas pastas de api/src. Setas tracejadas: dependência; triângulo vazado: realização da interface.

5.3.3 Diagrama de componentes (UML)

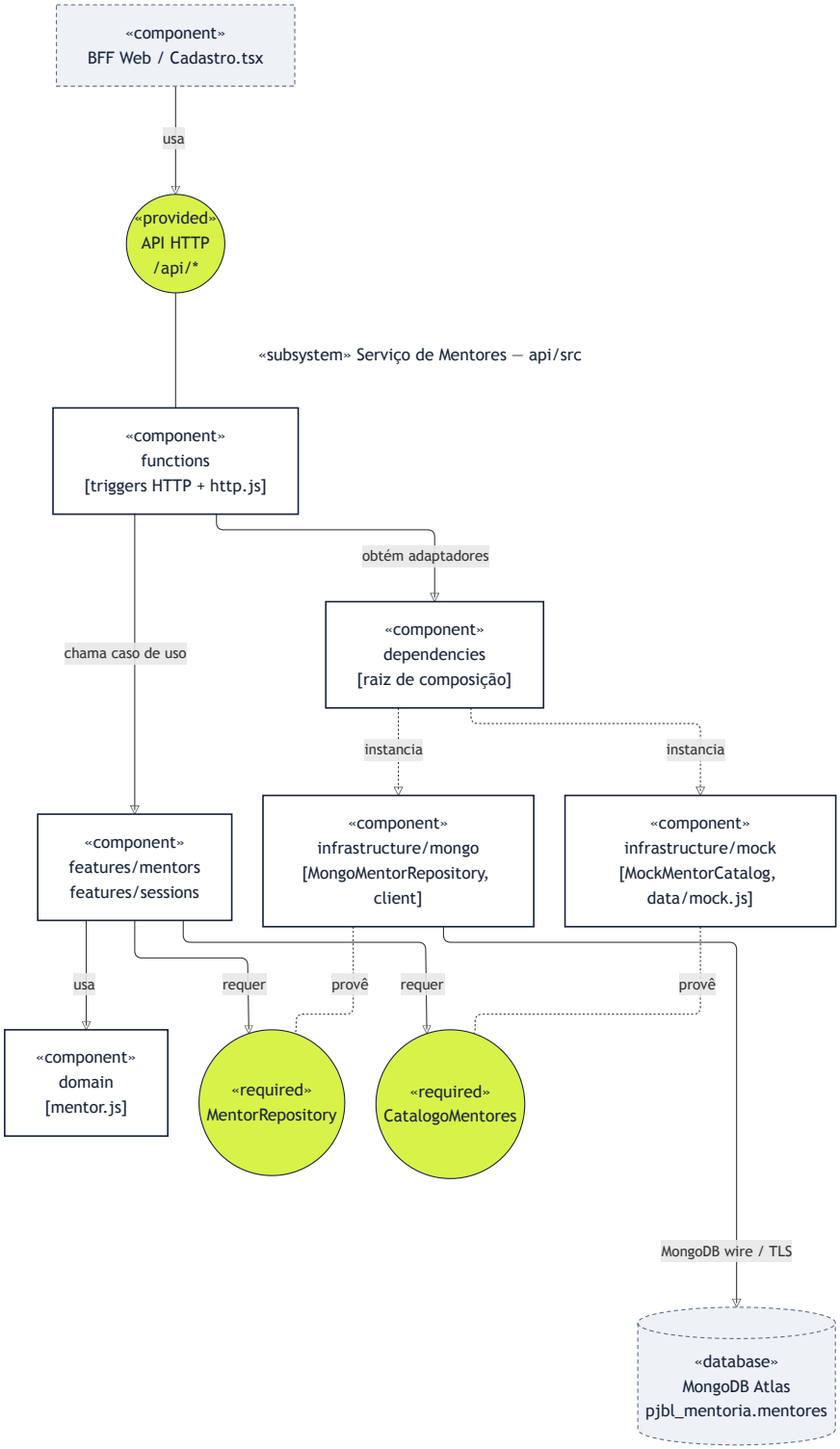


Figura 9. C4 Model — Nível 4 (Código). Diagrama de componentes UML com interfaces providas e requeridas. A fatia requer as portas; os adaptadores as proveem.

A notação mostra a inversão de dependência: `features` requer `MentorRepository` e `CatalogoMentores`, e quem as provê são os componentes de `infrastructure`. A seta de `infrastructure` aponta para a interface, nunca o contrário — é por isso que o mock pôde conviver com o MongoDB sem nenhuma alteração nas fatias de cadastro.

6. Visão de Runtime

> Como os blocos colaboram em execução. O primeiro cenário é código que roda hoje; o segundo é o caminho crítico do produto na arquitetura-alvo, com gateway, BFF e eventos.

6.1 Cenário implementado: alterar um mentor (CRUD no MongoDB)

O diagrama segue a chamada real, função por função. Os três desfechos de erro (400 por id inválido, 400 por validação, 404 por inexistente) são decididos dentro da fatia, e o 500 é decidido em `http.js`.

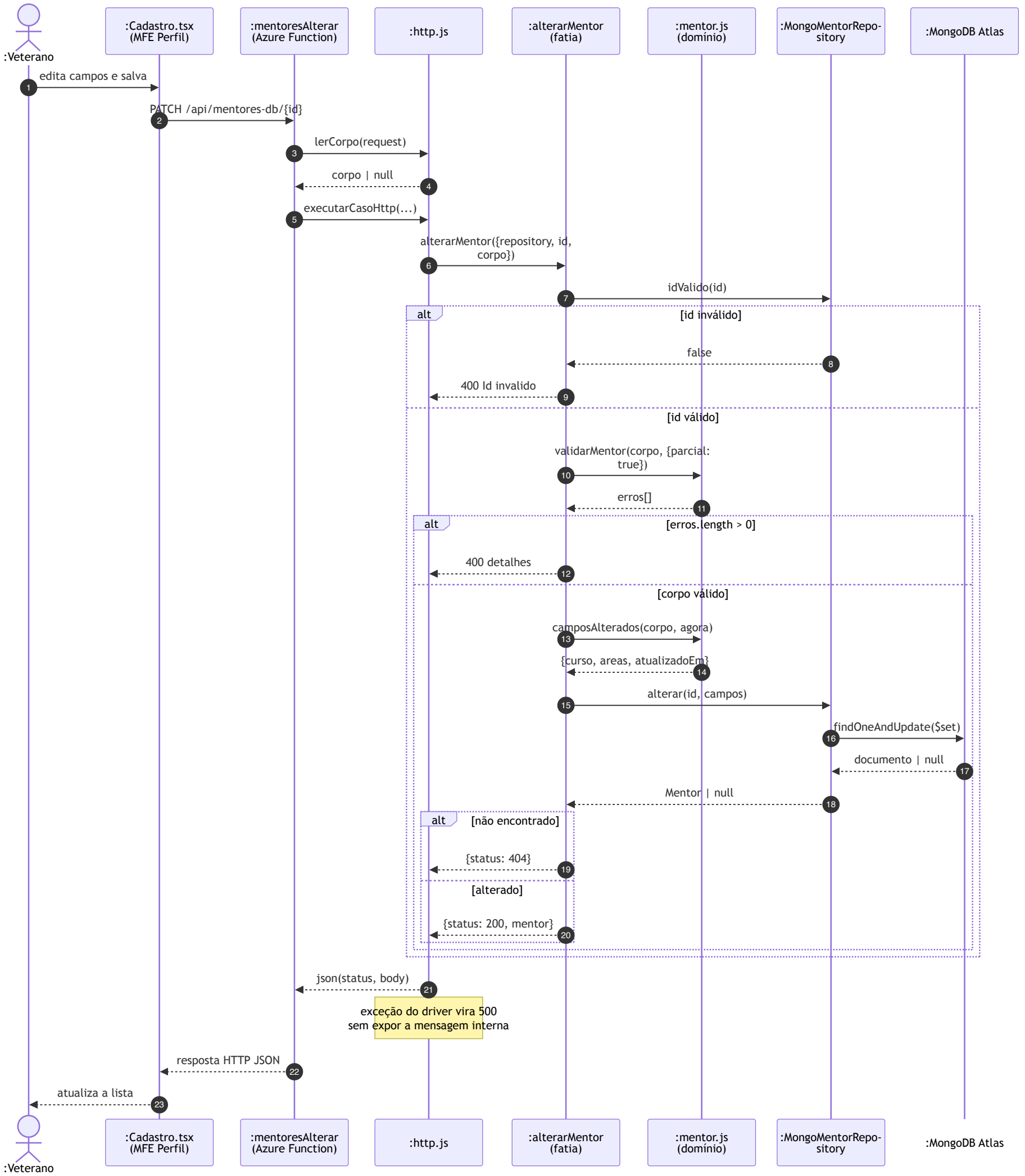


Figura 10. C4 Model — Nível 4 (Código). Diagrama de sequência UML de PATCH /api/mentores-db/:id, como implementado em api/src.

6.2 Cenário alvo: solicitar e aceitar uma sessão (RF6, RF7, RF8)

- **O gateway barra antes.** Token inválido ou excesso de chamadas é recusado no APIM, sem executar função.
- **A regra fica no dono.** Só o Serviço de Sessões decide se o horário é válido (422) e se o ator pode aceitar (403).
- **Gravar e avisar não dependem um do outro.** A sessão e o evento entram no banco na mesma transação (outbox); o relay publica depois. Se o Service Bus cair, a sessão já existe e o evento sai quando ele voltar.
- **O consumidor é idempotente.** Entrega “pelo menos uma vez” pode repetir a mensagem; o Serviço de Notificações descarta o `messageId` já processado.

A requisição do calouro termina no passo do `COMMIT`: a partir daí, a notificação é trabalho assíncrono e não afeta o tempo de resposta nem o resultado da solicitação.

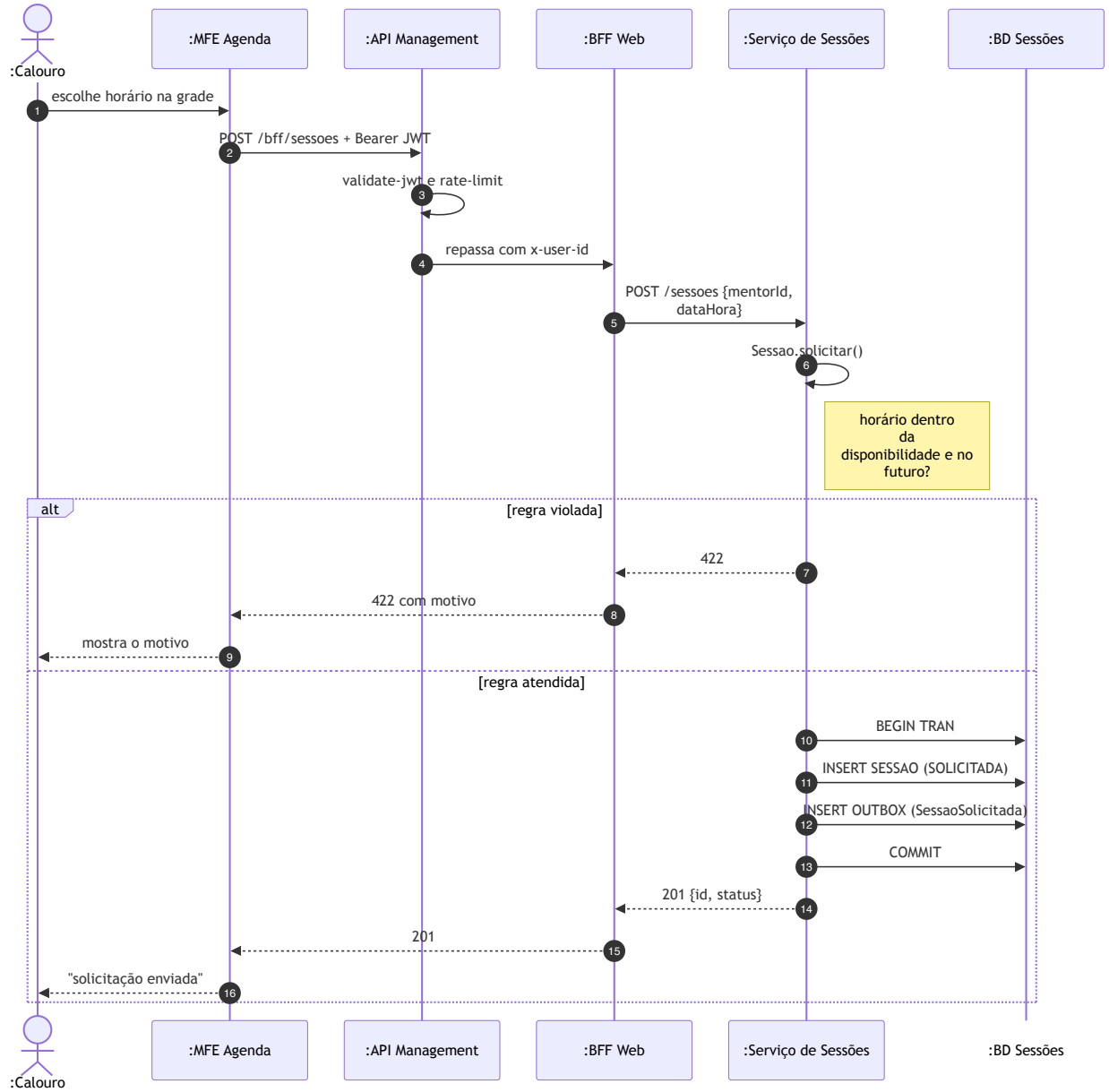


Figura 11. C4 Model — Nível 4 (Código). Diagrama de sequência UML, parte 1: o calouro solicita a sessão; gateway, BFF e Serviço de Sessões, com sessão e evento gravados na mesma transação (outbox).

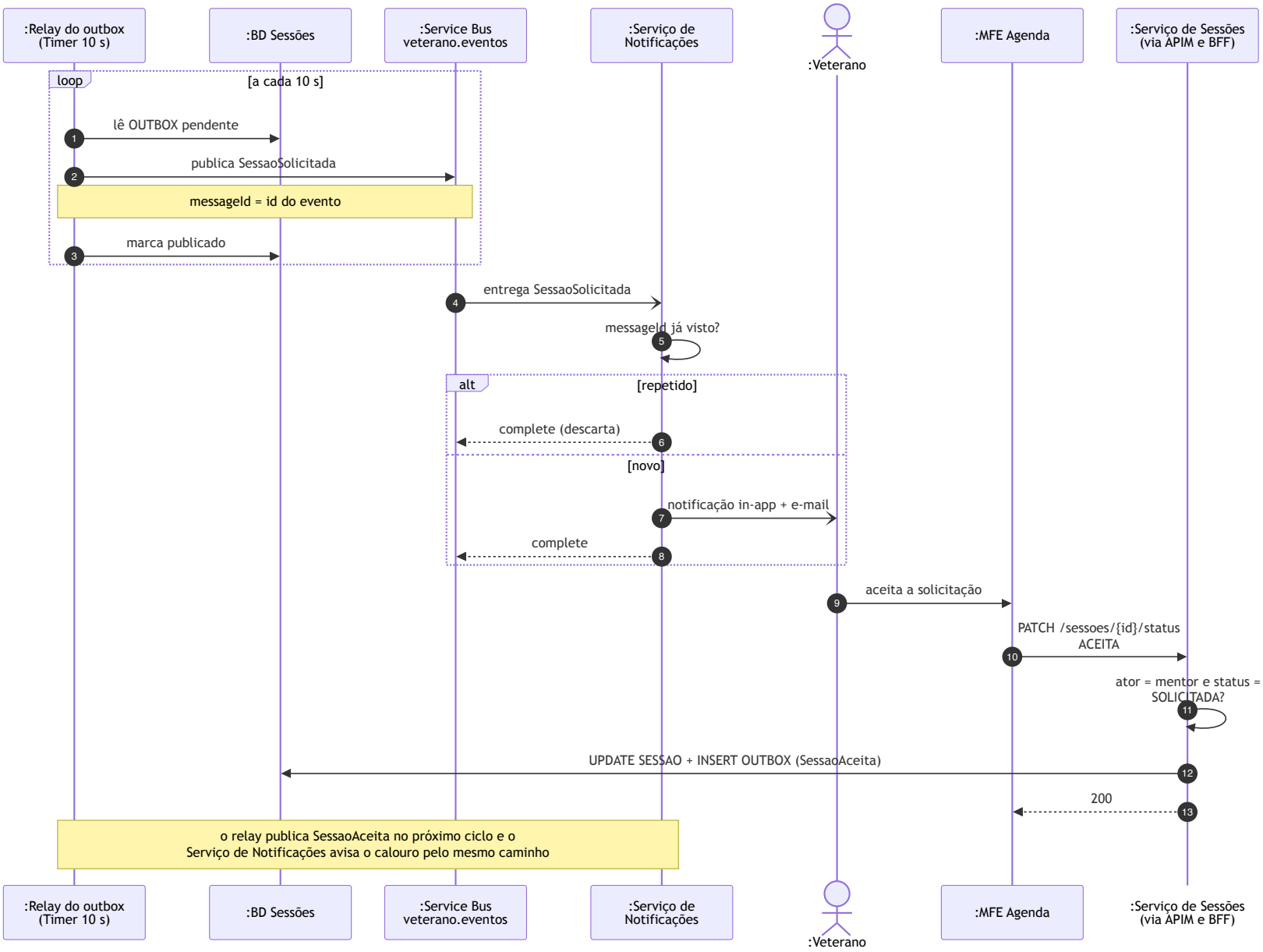


Figura 12. C4 Model — Nível 4 (Código). Diagrama de sequência UML, parte 2: relay publica o evento, o Serviço de Notificações consome de forma idempotente e o veterano aceita a sessão.

6.3 Ciclo de vida da Sessão

AÇÃO	QUEM PODE	A PARTIR DE	REGRA E EVENTO PUBLICADO
Solicitar	Mentorado	—	Horário dentro da disponibilidade e no futuro; não consigo mesmo. `SessaoSolicitada`.
Aceitar	Mentor da sessão	SOLICITADA	`SessaoAceita`.
Recusar	Mentor da sessão	SOLICITADA	Terminal. `SessaoRecusada`.
Cancelar	Qualquer participante	SOLICITADA ou ACEITA	Terminal. `SessaoCancelada`.
Concluir	Qualquer participante	ACEITA	`SessaoConcluida` — libera a avaliação no Serviço de Avaliações.

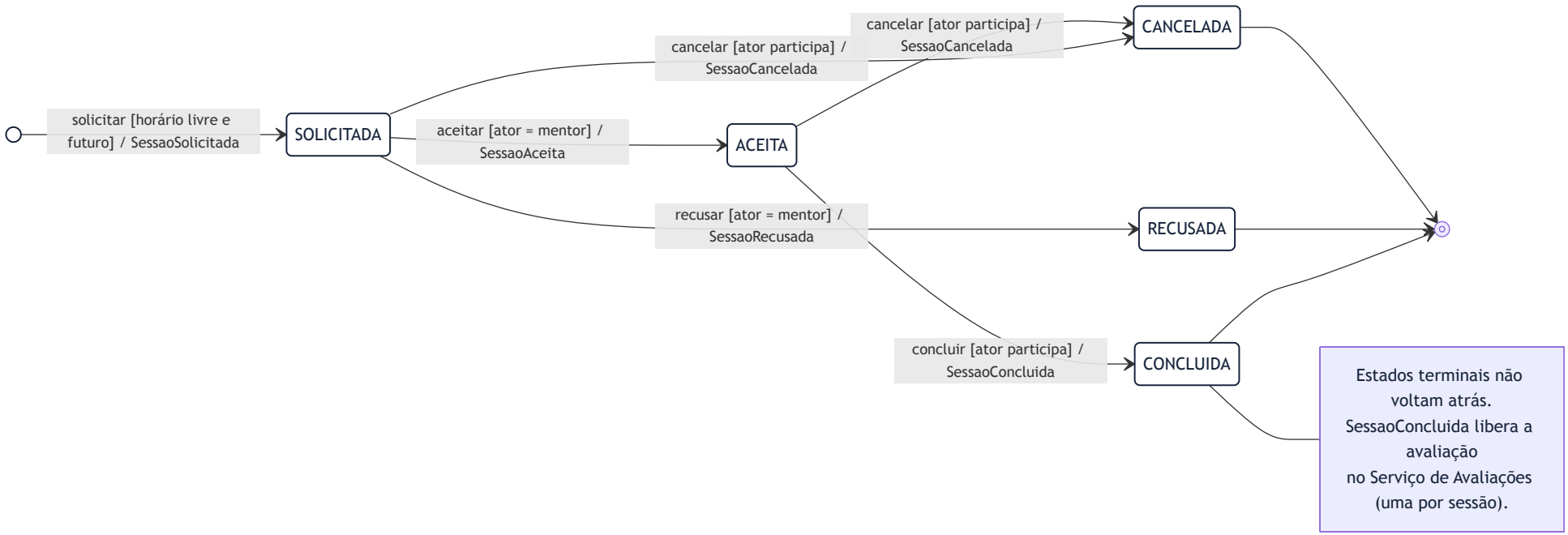


Figura 13. Diagrama de máquina de estados UML da Sessão. Notação: evento [guarda] / evento de domínio publicado.

6.4 Cenário alvo: avaliação atualiza a reputação

1. O Serviço de Avaliações recebe a nota, confere que a sessão está liberada e grava no Cosmos DB.
2. Publica `AvaliacaoRegistrada {mentorId, nota, avaliacaoId}` em `veterano.eventos`.
3. A assinatura `reputacao` entrega o evento ao Serviço de Mentores, que recalcula `REPUTACAO_PROJECAO` no documento do mentor usando a mesma `calcularReputacao` do domínio e guarda o último `avaliacaoId` aplicado (idempotência).
4. A busca de mentores lê a projeção local — sem consultar o banco de Avaliações. A reputação fica alguns segundos atrasada em relação à nota: consistência eventual aceita (ADR-06).

7. Visão de Implantação

> Infraestrutura de execução e o mapeamento dos contêineres sobre ela. As tabelas listam os recursos, a região e o pipeline; o diagrama de implantação UML (Figura ao final da seção) mostra os nós e os artefatos — em verde-limão, o que já está provisionado.

7.1 Recursos

RECURSO	NOME	SKU / PLANO	PAPEL
Static Web App (Shell)	`swa-pjbl-mentoria-7205`	Free	Serve a SPA atual; será o Shell. **IMPLEMENTADO**
Static Web Apps (MFEs)	`swa-mfe-*`	Free	Um por remote. PLANEJADO
Function App	`func-pjbl-mentoria`	Flex Consumption, Node 22	Serviço de Mentores e endpoints de tela. **IMPLEMENTADO**
Function Apps	`func-bff`, `func-sessoes`, `func-avaliacoes`, `func-notificacoes`	Flex Consumption	Um por serviço. PLANEJADO
MongoDB Atlas	cluster M0, banco `pjbl_mentoria`	Free (512 MB)	BD Mentores. **IMPLEMENTADO**
Storage Account	`stpjblmentoria7205`	Standard LRS	Pacotes de deploy; tabela de notificações. **IMPLEMENTADO**
Application Insights	`func-pjbl-mentoria`	Pago por GB	Telemetria. **IMPLEMENTADO**
API Management	`apim-veterano`	Consumption	API Gateway; cota gratuita mensal de chamadas. PLANEJADO
Service Bus	`sb-veterano`	Standard	Tópicos exigem Standard (o Basic só tem filas). PLANEJADO
Azure SQL Database	`sql-veterano/sessoes`	Serverless, auto-pause	BD Sessões. PLANEJADO
Cosmos DB	`cosmos-veterano`	NoSQL serverless	BD Avaliações. PLANEJADO
Key Vault	`kv-veterano`	Standard	Segredos via Managed Identity. PLANEJADO

7.2 Região

Tudo em **`centralus`**. `brazilsouth` está bloqueada pela policy da assinatura; entre as regiões liberadas, `centralus` suporta todos os serviços usados, inclusive Static Web Apps, o que evita tráfego entre regiões. O custo é latência maior para o usuário no Brasil (seção 11).

7.3 Pipeline e implantação independente

ARTEFATO	ORIGEM	COMO É PUBLICADO
Shell e cada MFE	`dist/` de cada projeto Vite	Hoje: `swa deploy` com token. Alvo: um workflow GitHub Actions por MFE, disparado só por mudanças na sua pasta.
Cada serviço	pacote zip da Function App	Hoje: `az functionapp deployment source config-zip`. Alvo: um workflow por serviço.
Políticas do gateway	XML versionado	Publicadas com a definição OpenAPI do BFF.
Infraestrutura	`infra.sh`	Script Azure CLI; migração para Bicep prevista.

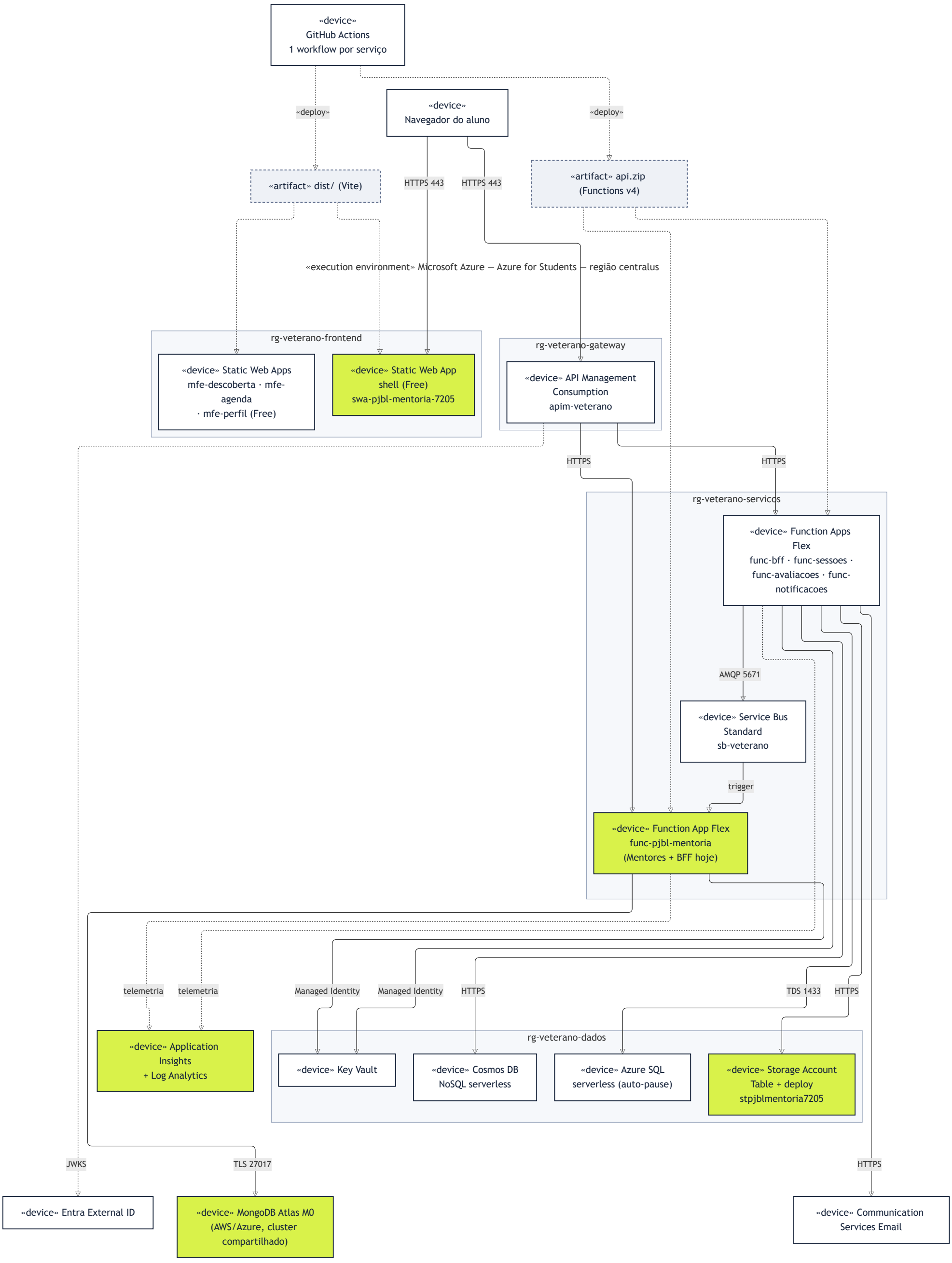


Figura 14. Diagrama de implantação UML. Nós de execução em Azure e externos, artefatos publicados e protocolos. Verde-limão: recurso já provisionado.

8. Conceitos Transversais

8.1 Arquitetura orientada a eventos

Toda comunicação assíncrona entre serviços passa por um único tópico do Service Bus. Cada consumidor tem sua assinatura com filtro, então recebe só os tipos de evento que lhe interessam e falha sem afetar os demais.

- **Envelope:** CloudEvents 1.0 (`id` , `type` , `source` , `time` , `data`). O `id` vira `messageId` no Service Bus e é a chave de idempotência.
- **Garantia:** pelo menos uma vez. O outbox impede perder evento; o consumidor idempotente impede efeito duplicado.
- **Ordem:** eventos da mesma sessão usam `sessionId = sessaoId` , preservando a ordem SOLICITADA → ACEITA → CONCLUIDA.
- **Falha:** após 5 tentativas a mensagem vai para a dead-letter da assinatura, com alerta no Application Insights.
- **Evolução:** campos só são adicionados; remover ou mudar significado exige novo `type` (`SessaoAceita.v2`).

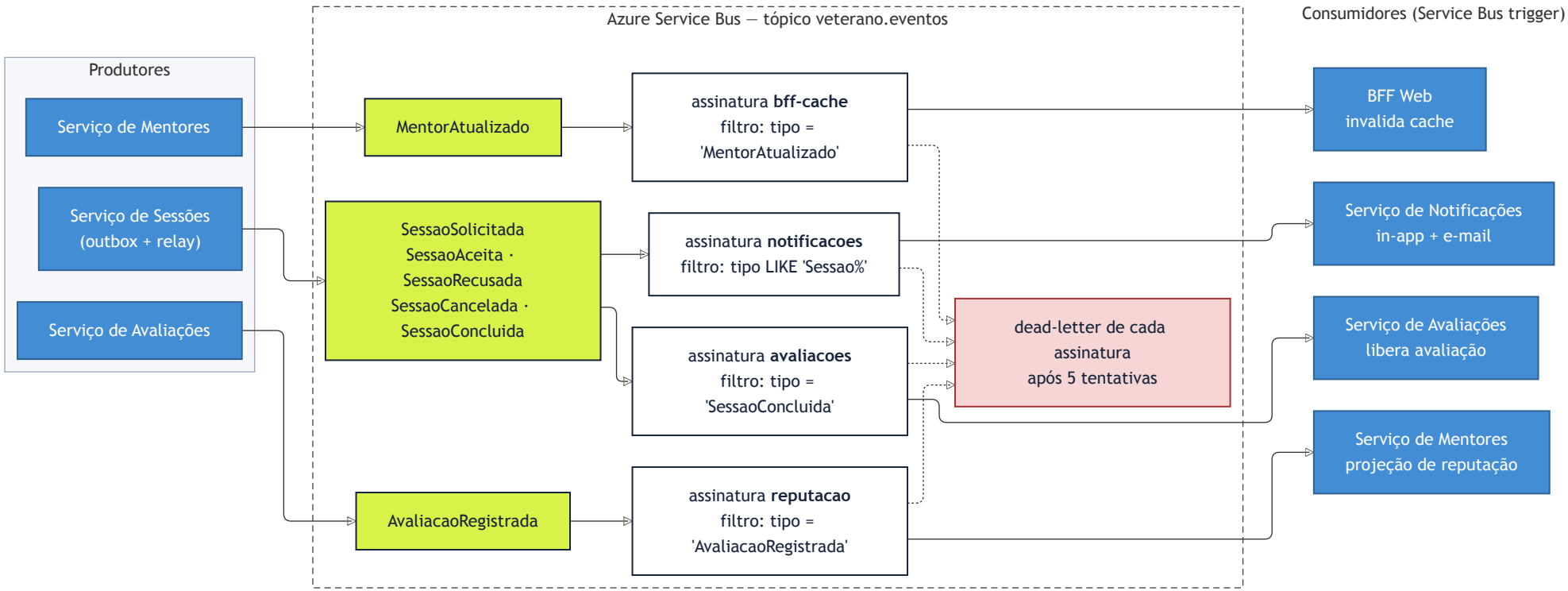


Figura 15. Fluxo de eventos de domínio: produtores, tópico veterano.eventos com assinaturas filtradas, consumidores e dead-letter.

EVENTO	PRODUTOR	CONSUMIDORES	PAYLOAD (CAMPO `data`)
`SessaoSolicitada`	Sessões	Notificações	`sessaoId, mentorId, mentoradoId, dataHora`
`SessaoAceita` / `Recusada` / `Cancelada`	Sessões	Notificações	`sessaoId, ator, statusAnterior`
`SessaoConcluida`	Sessões	Notificações, Avaliações	`sessaoId, mentorId, mentoradoId`
`AvaliacaoRegistrada`	Avaliações	Mentores	`avaliacaoId, sessaoId, mentorId, nota`
`MentorAtualizado`	Mentores	BFF (cache)	`mentorId, camposAlterados`

8.2 Dados: Database per Service — Diagrama de Entidades e Relacionamentos

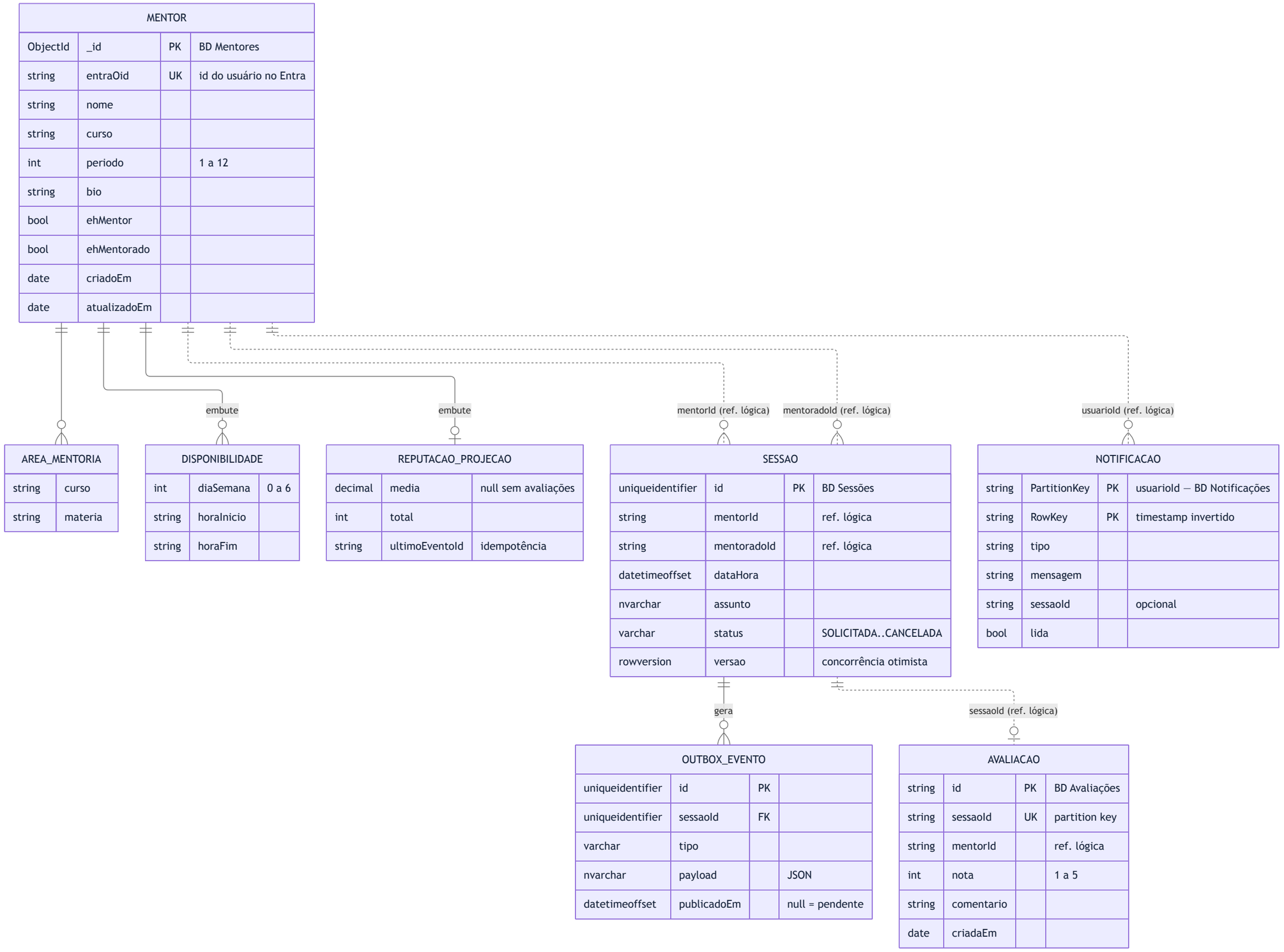


Figura 16. Diagrama de Entidades e Relacionamentos. Linha contínua: relacionamento dentro do mesmo banco. Linha tracejada: referência lógica entre bancos de serviços diferentes, por identificador, sem chave estrangeira.

BANCO	ENTIDADES	DECISÃO DE MODELAGEM
BD Mentores MongoDB Atlas	MENTOR com AREA_MENTORIA, DISPONIBILIDADE e REPUTACAO_PROJECAO embutidos	O perfil é lido inteiro de uma vez; documento único evita junção. Coleção real: `pjbl_mentoria.mentores`, com `nome`, `curso`, `periodo`, `bio`, `areas`, `criadoEm`, `atualizadoEm` (gravados por `novoMentor` / `camposAlterados`). Um único documento para mentor e mentorado, com `ehMentor` / `ehMentorado`.
BD Sessões Azure SQL	SESSAO, OUTBOX_EVENTO	Transição de estado precisa de transação e concorrência otimista (`rowversion`): dois cliques simultâneos de aceitar e cancelar não podem ambos vencer.
BD Avaliações Cosmos DB	AVALIACAO	`sessaoId` como chave de partição e única: o banco garante uma avaliação por sessão.
BD Notificações Table Storage	NOTIFICACAO	Acesso sempre por usuário: `PartitionKey = usuarioId`, `RowKey` com timestamp invertido para listar as mais recentes primeiro.

Referências entre serviços são identificadores, nunca chaves estrangeiras. Se um serviço precisa de dado de outro, recebe por evento e guarda uma cópia (a projeção de reputação) ou pede pelo BFF na leitura.

8.3 Serverless

- **Sem estado em memória.** Nenhuma função guarda dado entre invocações; o único cache é a conexão com o MongoDB (`criarProvedorColecao`), que reabre sozinha se cair.
- **Escala a zero.** Functions Flex, APIM Consumption e bancos serverless não cobram ociosidade relevante; Azure SQL pausa sozinho.
- ***Cold start* em cadeia.** Gateway → BFF → serviço podem acordar em sequência. Mitigação: Flex Consumption e, se medido como problema, uma instância sempre pronta só no BFF.
- **Gatilhos como contrato.** HTTP para leitura e comando; Service Bus para eventos; Timer para o relay do outbox.

8.4 Segurança

- **Autenticação no gateway.** `validate-jwt` confere assinatura, emissor e audiência do Entra antes de qualquer função; o BFF recebe o usuário em cabeçalho definido pelo gateway.
- **Autorização no dono do dado.** Só o Serviço de Sessões sabe se o usuário participa da sessão; resposta 403 caso contrário.
- **Serviços internos fechados.** Function Apps dos serviços aceitam apenas o BFF (chave de função ou restrição de acesso), nunca o navegador.
- **Segredos fora do código.** `MONGODB_URI` em configuração da Function App hoje; Key Vault com Managed Identity no alvo.
- **Estado atual.** Endpoints ainda anônimos (`authLevel: anonymous`) e CORS restrito à origem do Static Web App e a `localhost:5173` — registrado como dívida.

8.5 Validação e tratamento de erros

- Validação de domínio em `validarMentor` (completa na criação, parcial na alteração).
- Códigos: `400` payload ou id inválido; `404` inexistente; `500` falha de infraestrutura, sem a mensagem do driver; no alvo, `403` ator sem permissão e `422` violação de regra de negócio.
- Toda tela trata carregando, vazio e erro; nunca tela em branco.

8.6 Observabilidade

Application Insights em todas as Function Apps. O cabeçalho W3C `traceparent` é propagado do gateway ao BFF e aos serviços, e copiado para as propriedades da mensagem no Service Bus, permitindo ver uma solicitação de sessão e a notificação que ela gerou na mesma transação distribuída. Logs registram operação e filtros, sem dado pessoal.

8.7 Contratos entre microfrontends

Os MFEs não se importam entre si: comunicam-se pela URL (rotas) e por eventos de navegador (`CustomEvent` no `window`) quando necessário. React, React Router e o pacote de API são `shared` singletons. Estilo visual vem de tokens CSS do Shell (`index.css`), para que os remotes pareçam um único produto.

8.8 Uso de IA generativa

Front-end, Functions, reorganização em fatias e esta documentação foram produzidos com assistência do Claude Code (com Azure MCP e Azure CLI). Prompts versionados em `Prompt.md` e `docs/TDE2-PROMPTS.md` . Todo código passou por revisão humana: regras conferidas contra o PRD, endpoints testados e telas navegadas no ambiente publicado.

9. Decisões de Arquitetura

> Registradas no formato ADR. Entram as decisões caras de tomar ou de reverter.

ADR-01 — Microfrontends com Module Federation, extraídos de um monólito modular

CONTEXTO

Quatro alunos trabalham no front-end ao mesmo tempo; o Static Web Apps só serve arquivos estáticos.

DECISÃO

Shell + três remotes compostos no navegador por Module Federation (`@originjs/vite-plugin-federation`). Construir primeiro como SPA única com uma pasta por área e extrair cada pasta quando a fronteira estiver estável.

CONSEQUÊNCIAS

+ deploy independente por área; fronteira de MFE igual à de serviço. **–** versões de React precisam ser compatíveis entre remotes; uma falha de carga de remote precisa de fallback no Shell.

ALTERNATIVAS

iframes (isolamento forte, UX ruim); composição no servidor (incompatível com SWA estático).

ADR-02 — Azure API Management Consumption como API Gateway

CONTEXTO

Autenticação, limite de chamadas e CORS se repetiriam em cada Function App.

DECISÃO

APIM no tier Consumption na frente do BFF, com políticas `validate-jwt` , `rate-limit-by-key` e `cors` .

CONSEQUÊNCIAS

+ preocupações transversais em um lugar; serverless, com cota gratuita mensal. **–** mais um salto de rede e um *cold start* próprio.

ALTERNATIVAS

Roteamento do próprio SWA (sem JWT nem rate limit configuráveis); Application Gateway (não é serverless, custo fixo).

ADR-03 — Um BFF para a Web

CONTEXTO

As telas precisam de dados de vários serviços; só existe um tipo de cliente (navegador).

DECISÃO

Um único BFF Web em Functions, um endpoint por tela, sem banco e sem regra de negócio.

CONSEQUÊNCIAS

+ navegador faz uma chamada por tela; serviços ficam com contratos genéricos. **–** o BFF pode virar gargalo de mudanças se acumular lógica — regra: lógica de negócio nunca entra no BFF. Um app mobile futuro teria seu próprio BFF.

ADR-04 — Microsserviços por subdomínio, com banco próprio e poliglota

CONTEXTO

Mentores, Sessões, Avaliações e Notificações têm dados de formas diferentes e mudam por motivos diferentes.

DECISÃO

Quatro serviços, cada um dono do seu banco: documento (Mentores), relacional (Sessões), NoSQL serverless (Avaliações), chave-valor (Notificações). Nenhum serviço acessa banco alheio.

CONSEQUÊNCIAS

+ deploy, escala e falha isolados; cada dado no armazenamento que lhe cabe. **–** sem junção entre serviços; consultas cruzadas via BFF ou projeção; quatro tecnologias de dados para operar.

ADR-05 — Service Bus com tópico único e outbox transacional

CONTEXTO

Notificação e reputação precisam acontecer depois de uma escrita em outro serviço, mesmo que o consumidor esteja fora.

DECISÃO

Tópico `veterano.eventos` no Service Bus Standard, uma assinatura filtrada por consumidor; produtores com outbox e relay; consumidores idempotentes.

CONSEQUÊNCIAS

+ entrega garantida, dead-letter, ordenação por sessão, retentativa automática. **–** custo fixo mensal do tier Standard; consistência eventual visível ao usuário.

ALTERNATIVAS

Event Grid (mais barato, sem sessões/ordenação); Storage Queue (sem publicação para vários consumidores).

ADR-06 — Reputação como projeção atualizada por evento

CONTEXTO

Na versão 1, a reputação era calculada lendo as avaliações. Com Database per Service, o Serviço de Mentores não pode ler o banco de Avaliações.

DECISÃO

O Serviço de Mentores mantém `REPUTACAO_PROJECAO`, recalculada a cada `AvaliacaoRegistrada` com a mesma função `calcularReputacao` que já existe no domínio. Substitui o ADR-04 da versão 1.

CONSEQUÊNCIAS

+ busca rápida, sem chamada a outro serviço. **–** atraso de segundos entre avaliar e ver a nova média.

ADR-07 — Clean Architecture com Vertical Slice dentro de cada serviço

CONTEXTO

Os handlers originais misturavam HTTP, regra e acesso a dados; a troca de mock por MongoDB exigiria reescrevê-los.

DECISÃO

Um arquivo por caso de uso em `features/`; regras em `domain/`; contratos em `ports/`; implementações em `infrastructure/`; Azure Functions só em `functions/`; composição em `dependencies.js`.

CONSEQUÊNCIAS

+ caso de uso fácil de achar e mudar; domínio testável sem Azure nem banco; extração de serviço = mover fatias. **–** mais arquivos; pequena duplicação entre fatias parecidas, aceita para não acoplá-las.

ADR-08 — Flex Consumption em vez de Consumption clássico

CONTEXTO

A Function App em Consumption Linux respondia 503 até no SCM; o plano tem fim de suporte anunciado.

DECISÃO

Flex Consumption com Node 22 para todas as Function Apps.

CONSEQUÊNCIAS

+ subiu na primeira tentativa, *cold start* menor. **–** publicação por `config-zip`, pois `az functionapp deploy` responde 415.

ADR-09 — MongoDB Atlas como banco do Serviço de Mentores

CONTEXTO

A entrega de CRUD pedia persistência real; o perfil de mentor é um documento com listas aninhadas.

DECISÃO

MongoDB Atlas M0, acessado pelo driver oficial atrás da porta `MentorRepository`.

CONSEQUÊNCIAS

+ gratuito, modelo de documento natural para o perfil. **–** sai do catálogo Azure; limites do M0. Migrar para Cosmos DB for MongoDB troca só a URI, pois o protocolo é compatível.

10. Requisitos de Qualidade

10.1 Árvore de qualidade

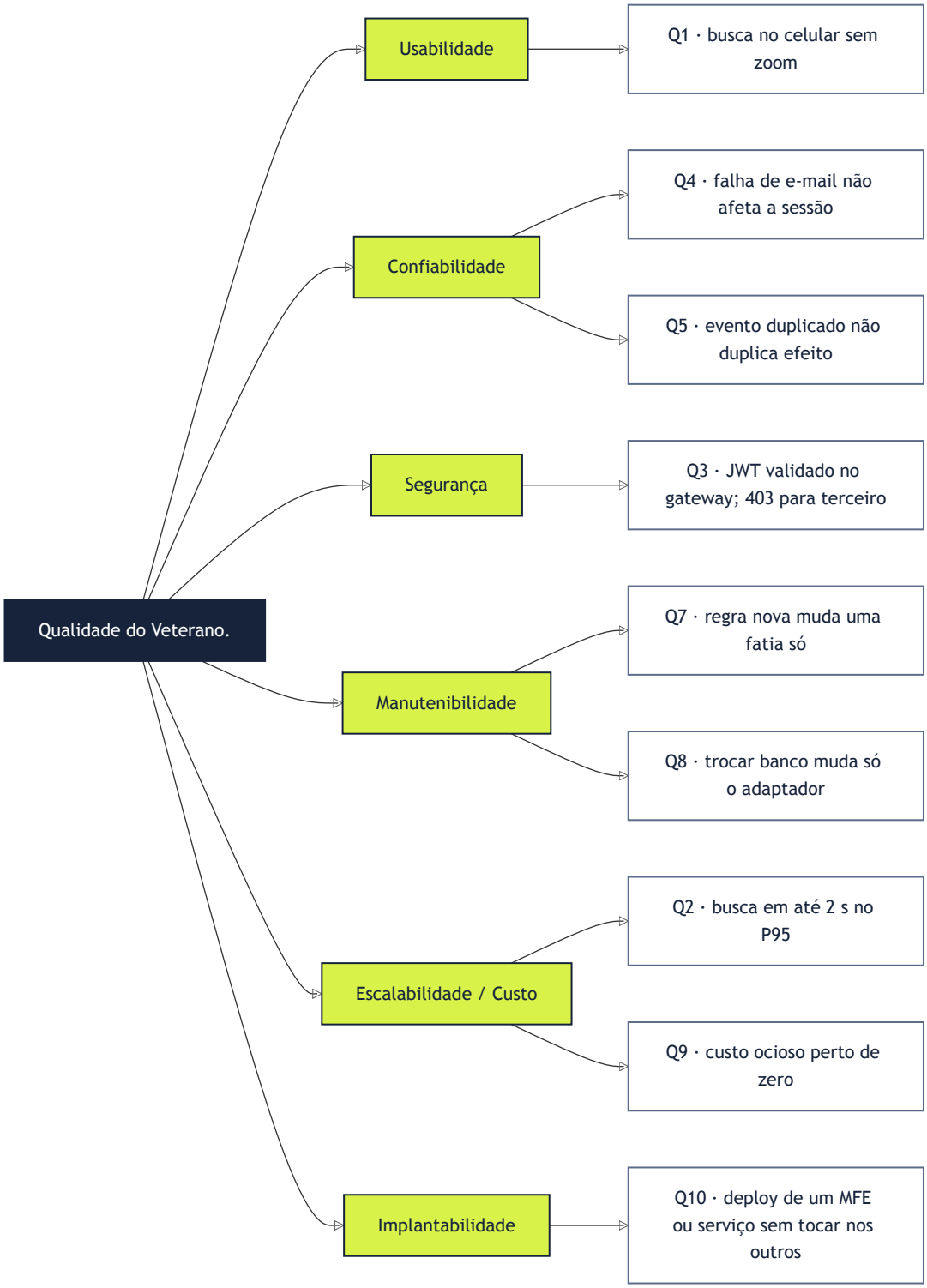


Figura 17. Árvore de requisitos de qualidade. As folhas referenciam os cenários da tabela 10.2.

10.2 Cenários de qualidade

#	ATRIBUTO	ESTÍMULO	RESPOSTA ESPERADA
Q1	Usabilidade	Calouro abre a plataforma no celular pela primeira vez.	Encontra e filtra mentores pela matéria sem zoom horizontal.
Q2	Desempenho	Busca com filtro de matéria.	Resposta em até 2 s no P95, descontado o *cold start*.
Q3	Segurança	Usuário altera sessão da qual não participa.	Token aceito no gateway, 403 no Serviço de Sessões, nada alterado, evento na telemetria.
Q4	Confiabilidade	ACS Email fora do ar durante um aceite.	Sessão aceita, notificação in-app gravada, e-mail registrado como aviso; resposta 200.
Q5	Confiabilidade	Service Bus entrega `SessaoAceita` duas vezes.	Uma única notificação, graças ao `messageId` já processado.
Q6	Corretude	Mentor sem avaliações aparece na busca.	“Sem avaliações ainda”, nunca 0.0; ordenado ao final (`media: null`).
Q7	Manutenibilidade	Mudar a regra de período válido do mentor.	Altera só `validarMentor` em `domain/mentor.js` ; triggers e adaptadores intactos.
Q8	Manutenibilidade	Trocar o catálogo mock por leitura no MongoDB.	Novo adaptador que cumpre `CatalogoMentores` e uma linha em `dependencies.js` .
Q9	Custo	Mês com uso esporádico.	Dentro do crédito; recursos serverless ociosos sem custo relevante.
Q10	Implantabilidade	Correção na tela de Agenda.	Só o MFE Agenda é publicado; Shell e demais remotes não mudam.

11. Riscos e Dívidas Técnicas

11.1 Riscos

RISCO	IMPACTO	MITIGAÇÃO
Complexidade distribuída para uma equipe de 4 pessoas	Alto	Extração incremental (ADR-01, ADR-07): só separa um serviço quando houver requisito que o exija; cada aluno dono de um par MFE + serviço.
Custo fixo de Service Bus Standard e APIM	Médio	APIM Consumption tem cota gratuita; Service Bus é o único custo fixo relevante. Alerta de orçamento na assinatura.
Cold start em cadeia gateway → BFF → serviço	Médio	Flex Consumption; instância pronta só no BFF se medição mostrar necessidade.
Consistência eventual percebida (reputação atrasada)	Baixo	Atraso de segundos; a tela de avaliação confirma o envio sem prometer média atualizada.
Latência de `centralus` para o Brasil	Médio	Estáticos pela borda do SWA; migrar quando a policy liberar `brazilsouth`.
Limites do MongoDB Atlas M0	Médio	Cliente reaproveitado; migração para Cosmos DB for MongoDB troca só a URI (ADR-09).

11.2 Dívidas técnicas assumidas

DÍVIDA	POR QUE FOI ACEITA	COMO SERÁ PAGA
Telas dos MFEs no mesmo bundle	Fronteiras ainda mudando.	Extrair cada pasta de `pages/` para um projeto Vite com Module Federation.
BFF e Serviço de Mentores na mesma Function App	Só existe um serviço de fato.	Separar quando o Serviço de Sessões existir e houver o que agregar.
Catálogo público e sessões sobre dados mock	Permitiu entregar telas antes dos bancos.	Novo adaptador para `CatalogoMentores` sobre MongoDB; sessões no futuro serviço.
Endpoints anônimos	Dados fictícios; identidade ainda não integrada.	APIM com `validate-jwt` e funções restritas ao BFF.
Sem testes automatizados	Verificação manual e `node --check`.	Testes de unidade das fatias com repositório falso — viável justamente porque as portas são injetadas.
Infra em script imperativo e deploy manual	Ferramental já instalado.	Bicep + um workflow GitHub Actions por MFE/serviço.

Apêndice — Glossário

TERMO	DEFINIÇÃO
Calouro / mentorado	Aluno que busca ajuda.
Veterano / mentor	Aluno que oferece mentoria. Os dois papéis podem coexistir na mesma pessoa.
Reputação	Média das notas do mentor com uma casa decimal, mais o total. Sem avaliações, `media = null`.
Fatia vertical (vertical slice)	Arquivo que implementa um caso de uso inteiro, do pedido à resposta, atravessando as camadas.
Porta / adaptador	Porta é o contrato que a aplicação exige; adaptador é a implementação concreta (MongoDB, mock, HTTP).
Raiz de composição	Único lugar que conhece as implementações concretas e as injeta (`dependencies.js`).
Microfrontend / Shell / remote	Parte do front-end publicada sozinha (remote), carregada em tempo de execução por uma aplicação hospedeira (Shell).
BFF	Backend for Frontend: camada que agrega serviços no formato de um cliente específico.
API Gateway	Ponto único de entrada que aplica autenticação, limites e roteamento antes dos serviços.
Database per Service	Cada microsserviço é o único dono e leitor do seu banco.
Outbox transacional	Tabela onde o evento é gravado na mesma transação do dado e publicado depois por um relay.
Idempotência	Processar a mesma mensagem duas vezes tem o mesmo efeito que processar uma.
Dead-letter	Fila para onde vão mensagens que falharam repetidamente, para análise.
Projeção	Cópia local de dado de outro serviço, mantida atualizada por eventos.
Cold start	Latência extra da primeira chamada a uma função sem instância ativa.
arc42 · C4 · DER	Template de documentação em seções; notação de diagramas em quatro níveis; diagrama de entidades e relacionamentos.

Documentação completa de arquitetura · Veterano. · versão 2.0 · outubro de 2026
Bento Barp · Guilherme Reis · Guilherme Selenko · Vinicius Trevisan
Estrutura conforme o template arc42 (arc42.org), traduzido e adaptado para o contexto do PJBL.